

model_testing

July 13, 2026

```
[3]: import os
      from matplotlib import figure
      from sklearn import linear_model

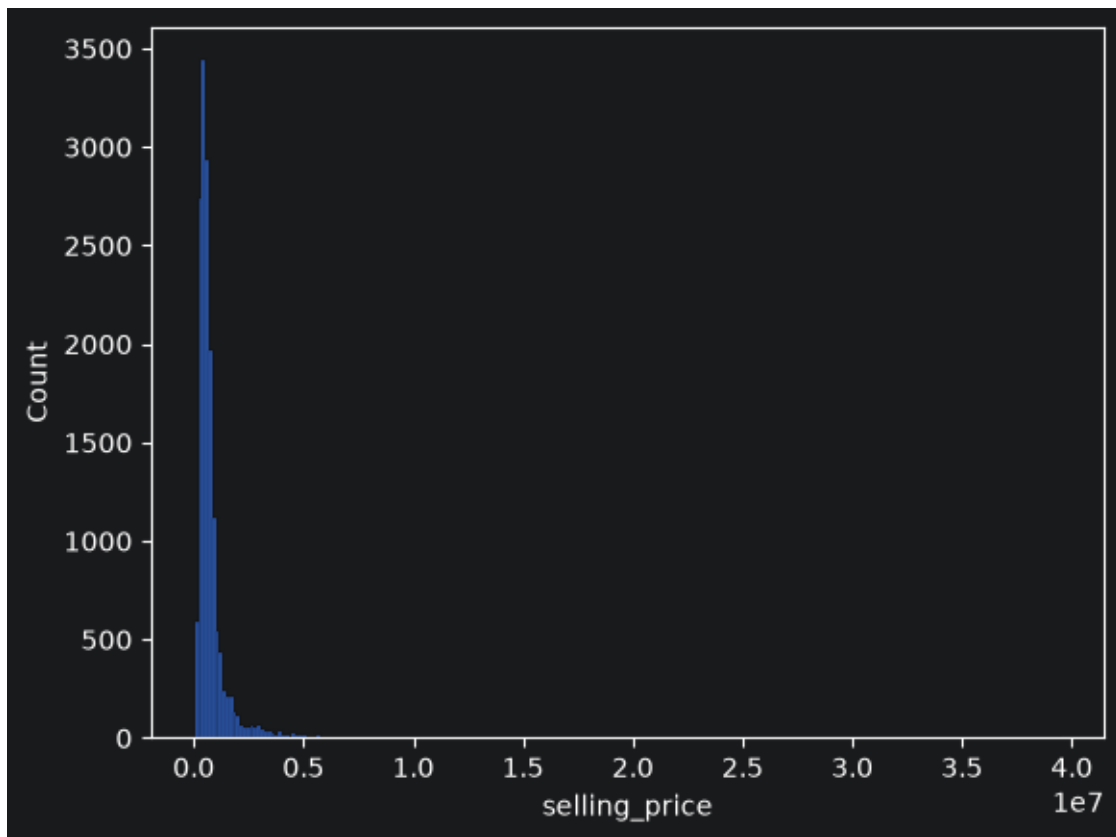
      print("Jupyter is looking here:", os.getcwd())
```

Jupyter is looking here: /home/owais/PycharmProjects/AUTOWORTH_BACKEND

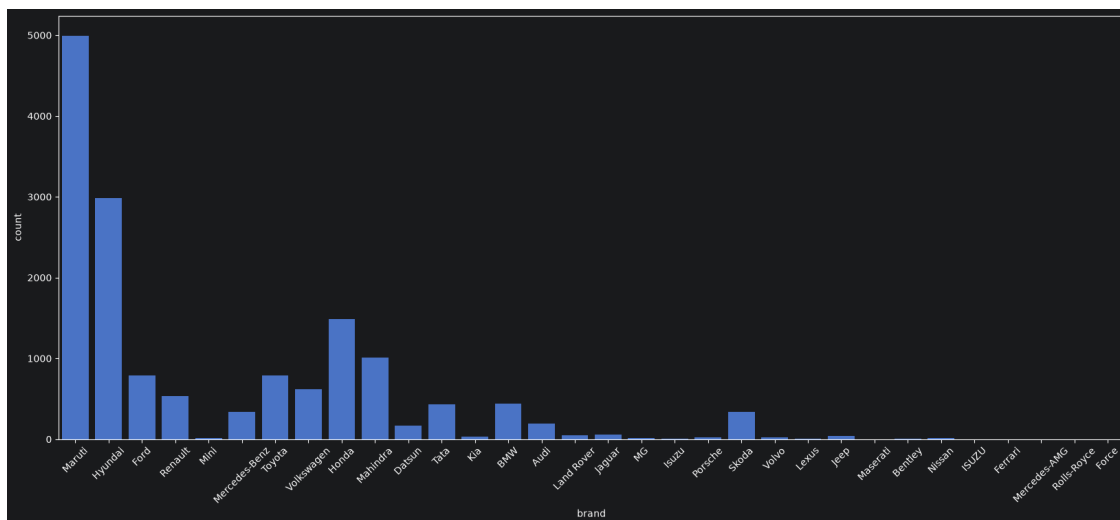
Visualizing Data Befor Starting Using Seabron

```
[4]: import pandas as pd
      import seaborn as sns
      df = pd.read_csv("car.csv")
      sns.histplot(df["selling_price"])
```

```
[4]: <Axes: xlabel='selling_price', ylabel='Count'>
```

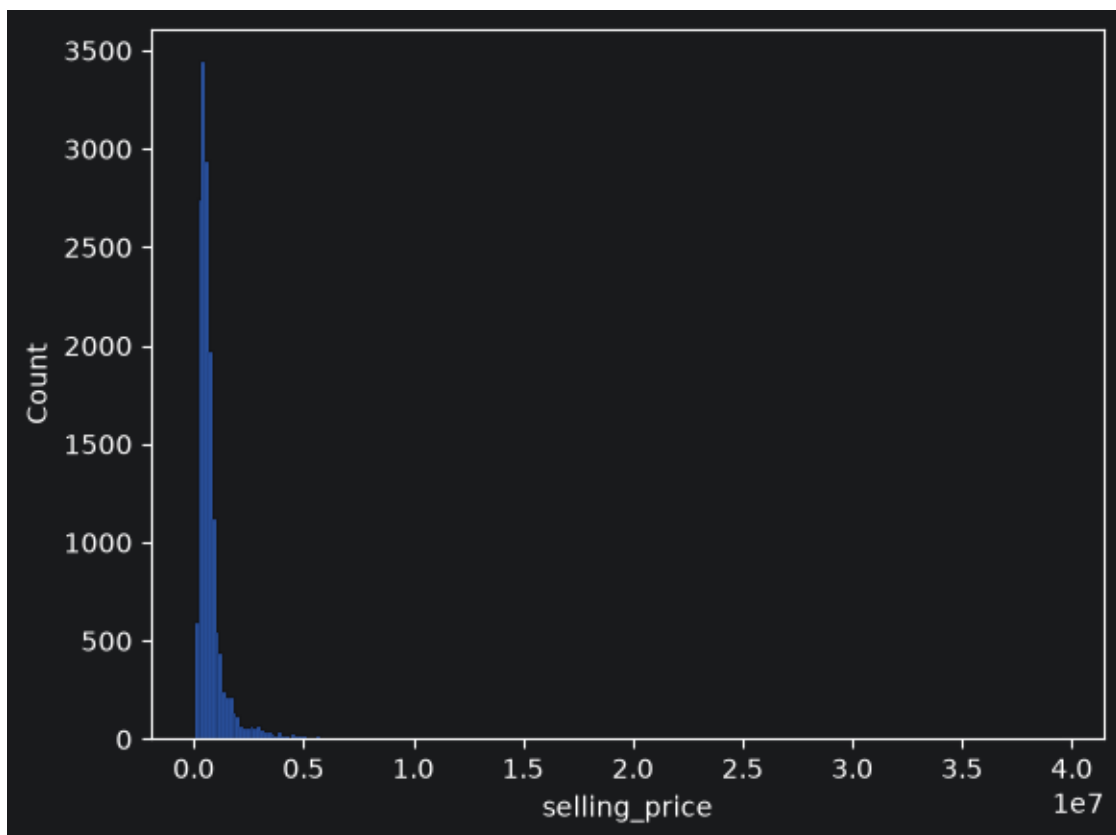


```
[5]: import matplotlib.pyplot as plt
plt.figure(figsize=(20,8))
sns.countplot(data=df, x="brand")
plt.xticks(rotation=45)
plt.show()
```



```
[6]: df["selling_price_lakh"] = df["selling_price"] / 100000
sns.histplot(df["selling_price"])
print(df["selling_price"])
```

```
0      120000
1     550000
2     215000
3     226000
4     570000
...
15406   250000
15407   925000
15408   425000
15409  1225000
15410  1200000
Name: selling_price, Length: 15411, dtype: int64
```



Checking for skewness so we can decide which Function Transformer to apply

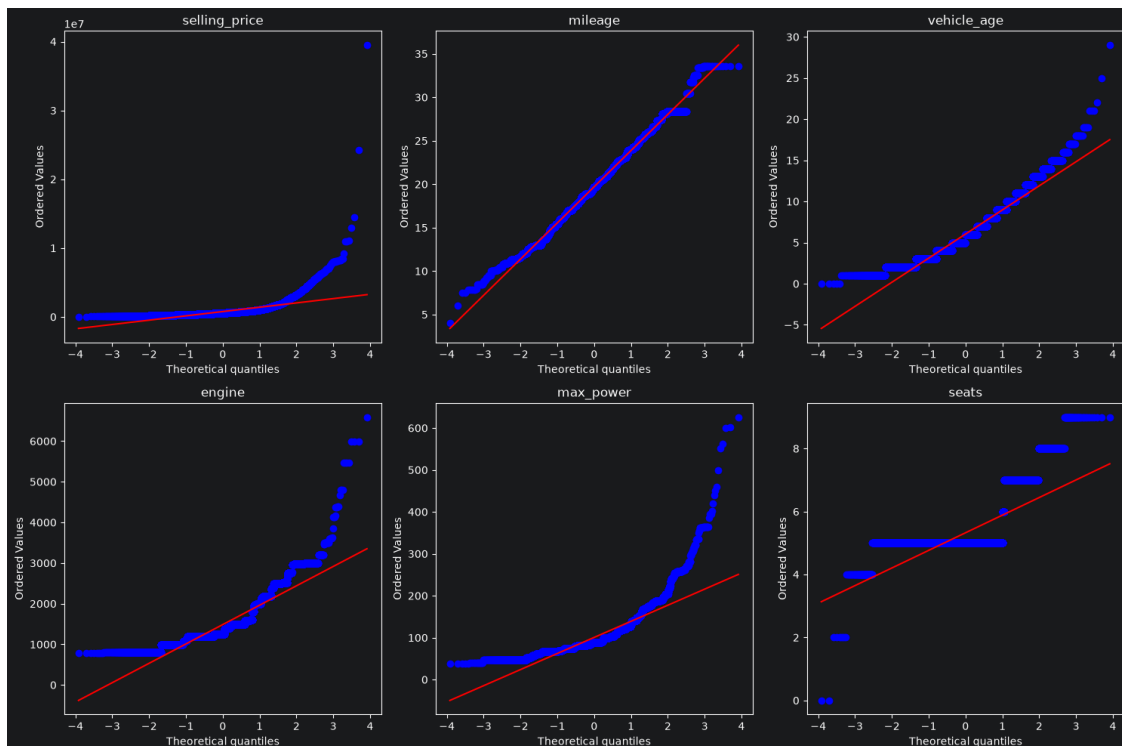
```
[7]: from scipy import stats
import matplotlib.pyplot as plt

columns = [
    "selling_price",
    "mileage",
    "vehicle_age",
    "engine",
    "max_power",
    "seats"
]

plt.figure(figsize=(15,10))

for i, col in enumerate(columns, 1):
    plt.subplot(2, 3, i)    # 2 rows, 3 columns
    stats.probplot(df[col], dist="norm", plot=plt)
    plt.title(col)

plt.tight_layout()
plt.show()
```



```
[8]: print(df[[
    "selling_price",
    "mileage",
    "vehicle_age",
    "engine",
    "max_power",
    "seats"
]].skew())
```

```
selling_price    10.047048
mileage          0.104961
vehicle_age      0.833712
engine           1.666467
max_power        2.485129
seats            2.039983
dtype: float64
```

AFTER ANALYZING DATA NOW DOING DATA CLEANING PROCESS

```
[9]: df.shape
```

```
[9]: (15411, 15)
```

```
[10]: df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 15411 entries, 0 to 15410
Data columns (total 15 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Unnamed: 0             15411 non-null  int64
1   car_name               15411 non-null  str
2   brand                  15411 non-null  str
3   model                  15411 non-null  str
4   vehicle_age            15411 non-null  int64
5   km_driven              15411 non-null  int64
6   seller_type            15411 non-null  str
7   fuel_type              15411 non-null  str
8   transmission_type      15411 non-null  str
9   mileage                15411 non-null  float64
10  engine                 15411 non-null  int64
11  max_power              15411 non-null  float64
12  seats                  15411 non-null  int64
13  selling_price           15411 non-null  int64
14  selling_price_lakh     15411 non-null  float64
dtypes: float64(3), int64(6), str(6)
memory usage: 1.8 MB
```

```
[11]: df.head()
```

```
[11]: Unnamed: 0      car_name  brand  model  vehicle_age  km_driven  \
0          0    Maruti Alto  Maruti   Alto           9    120000
1          1  Hyundai Grand  Hyundai  Grand           5     20000
2          2  Hyundai i20   Hyundai   i20          11    60000
3          3    Maruti Alto  Maruti   Alto           9    37000
4          4  Ford Ecosport   Ford  Ecosport           6    30000

      seller_type fuel_type transmission_type  mileage  engine  max_power  seats  \
0  Individual    Petrol           Manual    19.70    796      46.30     5
1  Individual    Petrol           Manual    18.90   1197      82.00     5
2  Individual    Petrol           Manual    17.00   1197      80.00     5
3  Individual    Petrol           Manual    20.92    998      67.10     5
4      Dealer    Diesel           Manual    22.77   1498      98.59     5

      selling_price  selling_price_lakh
0          120000          1.20
1          550000          5.50
2          215000          2.15
3          226000          2.26
4          570000          5.70
```

```
[12]: df.isnull().sum()
```

```
[12]: Unnamed: 0      0
car_name      0
brand         0
model         0
vehicle_age   0
km_driven     0
seller_type   0
fuel_type     0
transmission_type  0
mileage       0
engine        0
max_power     0
seats         0
selling_price  0
selling_price_lakh  0
dtype: int64
```

Identify Unique Values in Each Columnn

```
[13]: for col in ['brand', 'fuel_type', 'transmission_type', 'seller_type']:
      print(col, df[col].unique())
```

```
brand <StringArray>
[      'Maruti',      'Hyundai',      'Ford',      'Renault',
      'Mini', 'Mercedes-Benz',      'Toyota',      'Volkswagen',
```

```

        'Honda',      'Mahindra',      'Datsun',      'Tata',
        'Kia',        'BMW',        'Audi',        'Land Rover',
        'Jaguar',      'MG',        'Isuzu',        'Porsche',
        'Skoda',       'Volvo',     'Lexus',       'Jeep',
        'Maserati',    'Bentley',   'Nissan',       'ISUZU',
        'Ferrari',     'Mercedes-AMG', 'Rolls-Royce', 'Force']
Length: 32, dtype: str
fuel_type <StringArray>
['Petrol', 'Diesel', 'CNG', 'LPG', 'Electric']
Length: 5, dtype: str
transmission_type <StringArray>
['Manual', 'Automatic']
Length: 2, dtype: str
seller_type <StringArray>
['Individual', 'Dealer', 'Trustmark Dealer']
Length: 3, dtype: str
Chekcing For Duplicates

```

```

[14]: for col in df.columns:
        duplicates = df[col].duplicated().sum()
        print(f"{col}: {duplicates}")

```

```

Unnamed: 0: 0
car_name: 15290
brand: 15379
model: 15291
vehicle_age: 15387
km_driven: 11723
seller_type: 15408
fuel_type: 15406
transmission_type: 15409
mileage: 15000
engine: 15301
max_power: 15069
seats: 15403
selling_price: 14325
selling_price_lakh: 14325

```

```

[15]: df['model'].nunique()

df['vehicle_age'].nunique()

```

```

[15]: 24

```

```

[16]: df.groupby(['brand', 'model']).ngroups

```

```

[16]: 121

```

```
[17]: df.groupby(['brand', 'model', 'vehicle_age']).ngroups
```

```
[17]: 811
```

```
[18]: df.groupby(['brand', 'model', 'vehicle_age', 'km_driven']).ngroups
```

```
[18]: 11871
```

```
[19]: df.groupby(['brand', 'model', 'vehicle_age', 'km_driven', 'selling_price']).  
      ↪ngroups
```

```
[19]: 15085
```

```
[20]: df.duplicated().sum()
```

```
[20]: np.int64(0)
```

TRAINING MODELS AFTER THOROUGH DATA ANALYSIS

FIRST MODEL USING LINEAR REGRESSION

```
[21]: import pandas as pd  
      from sklearn.preprocessing import LabelEncoder  
      le=LabelEncoder()  
      df=pd.read_csv("car.csv")  
      df=pd.get_dummies(df,drop_first=True)  
      from sklearn.model_selection import train_test_split  
      from sklearn.linear_model import LinearRegression  
      from sklearn.metrics import r2_score  
      x=df.drop("selling_price",axis=1)  
      y=df[["selling_price"]]  
      model=LinearRegression()  
      X_train,X_test,y_train,y_test=train_test_split(x,y,test_size=0.  
      ↪2,random_state=42)  
      model.fit(X_train,y_train)  
      prediction=model.predict(X_test)  
      r2score=r2_score(y_test,prediction)  
      print(r2score)  
      df.head()
```

```
0.7406080169863216
```

```
[21]:   Unnamed: 0  vehicle_age  km_driven  mileage  engine  max_power  seats  \  
0          0           9    120000    19.70     796     46.30     5  
1          1           5     20000    18.90    1197     82.00     5  
2          2          11     60000    17.00    1197     80.00     5  
3          3           9     37000    20.92     998     67.10     5  
4          4           6     30000    22.77    1498     98.59     5
```


	selling_price	car_name_Audi A6	car_name_Audi A8	...	model_i10	\
0	120000	False	False	...	False	
1	550000	False	False	...	False	
2	215000	False	False	...	False	
3	226000	False	False	...	False	
4	570000	False	False	...	False	

	model_i20	model_redi-G0	seller_type_Individual	\
0	False	False	True	
1	False	False	True	
2	True	False	True	
3	False	False	True	
4	False	False	False	

	seller_type_Trustmark Dealer	fuel_type_Diesel	fuel_type_Electric	\
0	False	False	False	
1	False	False	False	
2	False	False	False	
3	False	False	False	
4	False	True	False	

	fuel_type_LPG	fuel_type_Petrol	transmission_type_Manual
0	False	True	True
1	False	True	True
2	False	True	True
3	False	True	True
4	False	False	True

[5 rows x 285 columns]

ABOVE MODEL GIVES RSQUARE SCORE OF 0.74 NW APPLYNG STANDARD SCALING AND TEHN RECALCULAITN R SQUARE

```
[22]: import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score
df = pd.read_csv("car.csv")
df = pd.get_dummies(df, drop_first=True)
# --- Separate features and target ---
x = df.drop("selling_price", axis=1)
y = df[["selling_price"]]
# --- Split first, THEN scale ---
X_train, X_test, y_train, y_test = train_test_split(x, y, test_size=0.
↪2, random_state=42)
```

```

# --- Scale input features only ---
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
# --- Train ---
model = LinearRegression()
model.fit(X_train, y_train)
# --- Evaluate ---
prediction = model.predict(X_test)
print("R² score:", round(r2_score(y_test, prediction), 4))

```

R² score: 0.7896

AFTER APPLYING ONEHOT ENCODING AND STANDARD SCALING SCORE IS 0.7896

```

[23]: import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score

df = pd.read_csv("car.csv")
df = pd.get_dummies(df, drop_first=True)

x = df.drop("selling_price", axis=1)
y = df[["selling_price"]]

X_train, X_test, y_train, y_test = train_test_split(x, y, test_size=0.2,
    random_state=42)

numeric_cols = ['vehicle_age', 'km_driven', 'mileage', 'engine', 'max_power',
    'seats']

# --- Scale ONLY those columns, leave onehot columns untouched ---
scaler = StandardScaler()
X_train[numeric_cols] = scaler.fit_transform(X_train[numeric_cols])
X_test[numeric_cols] = scaler.transform(X_test[numeric_cols])

model = LinearRegression()
model.fit(X_train, y_train)

prediction = model.predict(X_test)
print("R² score:", round(r2_score(y_test, prediction), 4))

```

R² score: 0.7995

CONCLUSION:-StandardScaler's conclusion: scale only genuine numeric columns, not onehot columns — smallbut real R^2 improvement, confirmed by your test (0.789→0.795)

AS IN BEGININ WE HAD ANALYZED THE SKEWNESS OF DATA USING PROB-PLOT/QQPLOT WE NEED TO APPLY FUNCTION TRANSFORMATIOSNS LIKE SQUARE TRANSFORM OR LOG TRANSFORM DEPENDING UPON THE DATA

```
[24]: import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_absolute_error
df = pd.read_csv("car.csv")

df = pd.get_dummies(df, drop_first=True)
X = df.drop("selling_price", axis=1)
y = df["selling_price"]
# log transform target
y_log = np.log1p(y)
X_train, X_test, y_train_log, y_test_log = train_test_split(
X, y_log, test_size=0.2, random_state=42
)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
model = LinearRegression()
model.fit(X_train, y_train_log)
# predict log price
pred_log = model.predict(X_test)
# convert back to actual price
pred_price = np.expm1(pred_log)
y_test_price = np.expm1(y_test_log)
print("R2 Score:", round(r2_score(y_test_price, pred_price), 4))
print("MAE:", round(mean_absolute_error(y_test_price, pred_price), 2))
```

R2 Score: 0.8907

MAE: 117887.35

I identified skewed columns using QQ plots, but I validated each transformation empirically by comparing R^2 before and after, rather than assuming distribution-shape fixes always help. Some transforms improved performance, others didn't, so I kept only the ones that measurably helped."

AFTER GETTING MAX R SQUARE OF 0.8907 now trying anotehr dataset wiht mro features

```
[25]: df2=pd.read_csv("delhicar.csv")
df2.head()
```

```
[25]:
```

	city	make	model	variant	mileage \
0	delhi-ncr	Hyundai	New i20 Sportz	1.2 MT Dual Tone	80637
1	delhi-ncr	Maruti Suzuki	Vitara Brezza	VXi	41189
2	delhi-ncr	Honda	Amaze	1.2 S MT Petrol	17293
3	delhi-ncr	Datsun	Redi-GO	T	43941
4	delhi-ncr	Hyundai	Creta	1.6 SX Plus Auto Petrol	66029

	make_year	price	fuel_type	no_of_owners	color	body_type \
0	2021	556000.0	petrol	1	silver	hatchback
1	2021	630000.0	petrol	2	red	suv
2	2018	506000.0	petrol	1	red	sedan
3	2016	182000.0	petrol	2	grey	hatchback
4	2017	788000.0	petrol	1	white	suv

	transmission	registration_year	latest_publish_date
0	manual	2021	2025-08-18 12:35:31
1	manual	2021	2025-08-18 12:40:34
2	manual	2018	2025-08-18 12:50:12
3	manual	2018	2025-08-18 10:45:45
4	automatic	2017	2025-08-18 12:57:20

CHECK MISSING VALUES

```
[26]: df.isnull().sum()
```

```
[26]: city          0
make              0
model            0
variant          0
mileage          0
make_year        0
price            0
fuel_type        0
no_of_owners     0
color            1
body_type        0
transmission     0
registration_year 0
latest_publish_date 0
dtype: int64
```

```
[27]: df.duplicated().sum()
df[df.duplicated()]
```

```
[27]: Empty DataFrame
Columns: [Unnamed: 0, vehicle_age, km_driven, mileage, engine, max_power, seats,
selling_price, car_name_Audi A6, car_name_Audi A8, car_name_Audi Q7,
car_name_BMW 3, car_name_BMW 5, car_name_BMW 6, car_name_BMW 7, car_name_BMW X1,
```

```

car_name_BMW X3, car_name_BMW X4, car_name_BMW X5, car_name_BMW Z4,
car_name_Bentley Continental, car_name_Datsun GO, car_name_Datsun RediGO,
car_name_Datsun redi-GO, car_name_Ferrari GTC4Lusso, car_name_Force Gurkha,
car_name_Ford Aspire, car_name_Ford Ecosport, car_name_Ford Endeavour,
car_name_Ford Figo, car_name_Ford Freestyle, car_name_Honda Amaze,
car_name_Honda CR, car_name_Honda CR-V, car_name_Honda City, car_name_Honda
Civic, car_name_Honda Jazz, car_name_Honda WR-V, car_name_Hyundai Aura,
car_name_Hyundai Creta, car_name_Hyundai Elantra, car_name_Hyundai Grand,
car_name_Hyundai Santro, car_name_Hyundai Tucson, car_name_Hyundai Venue,
car_name_Hyundai Verna, car_name_Hyundai i10, car_name_Hyundai i20,
car_name_ISUZU MUX, car_name_Isuzu D-Max, car_name_Isuzu MUX, car_name_Jaguar
F-PACE, car_name_Jaguar XE, car_name_Jaguar XF, car_name_Jeep Compass,
car_name_Jeep Wrangler, car_name_Kia Carnival, car_name_Kia Seltos,
car_name_Land Rover Rover, car_name_Lexus ES, car_name_Lexus NX, car_name_Lexus
RX, car_name_MG Hector, car_name_Mahindra Alturas, car_name_Mahindra Bolero,
car_name_Mahindra KUV, car_name_Mahindra KUV100, car_name_Mahindra Marazzo,
car_name_Mahindra Scorpio, car_name_Mahindra Thar, car_name_Mahindra XUV300,
car_name_Mahindra XUV500, car_name_Maruti Alto, car_name_Maruti Baleno,
car_name_Maruti Celerio, car_name_Maruti Ciaz, car_name_Maruti Dzire LXI,
car_name_Maruti Dzire VXi, car_name_Maruti Dzire ZXI, car_name_Maruti Eeco,
car_name_Maruti Ertiga, car_name_Maruti Ignis, car_name_Maruti S-Presso,
car_name_Maruti Swift, car_name_Maruti Swift Dzire, car_name_Maruti Vitara,
car_name_Maruti Wagon R, car_name_Maruti XL6, car_name_Maserati Ghibli,
car_name_Maserati Quattroporte, car_name_Mercedes-AMG C, car_name_Mercedes-Benz
C-Class, car_name_Mercedes-Benz CLS, car_name_Mercedes-Benz E-Class,
car_name_Mercedes-Benz GL-Class, car_name_Mercedes-Benz GLS, car_name_Mercedes-
Benz S-Class, car_name_Mini Cooper, car_name_Nissan Kicks, car_name_Nissan
X-Trail, ...]
Index: []

```

```
[0 rows x 285 columns]
```

```
[28]: for col in df.columns:
        print(col, df[col].duplicated().sum())
```

```

Unnamed: 0 0
vehicle_age 15387
km_driven 11723
mileage 15000
engine 15301
max_power 15069
seats 15403
selling_price 14325
car_name_Audi A6 15409
car_name_Audi A8 15409
car_name_Audi Q7 15409
car_name_BMW 3 15409
car_name_BMW 5 15409

```

car_name_BMW 6 15409
car_name_BMW 7 15409
car_name_BMW X1 15409
car_name_BMW X3 15409
car_name_BMW X4 15409
car_name_BMW X5 15409
car_name_BMW Z4 15409
car_name_Bentley Continental 15409
car_name_Datsun GO 15409
car_name_Datsun RediGO 15409
car_name_Datsun redi-GO 15409
car_name_Ferrari GTC4Lusso 15409
car_name_Force Gurkha 15409
car_name_Ford Aspire 15409
car_name_Ford Ecosport 15409
car_name_Ford Endeavour 15409
car_name_Ford Figo 15409
car_name_Ford Freestyle 15409
car_name_Honda Amaze 15409
car_name_Honda CR 15409
car_name_Honda CR-V 15409
car_name_Honda City 15409
car_name_Honda Civic 15409
car_name_Honda Jazz 15409
car_name_Honda WR-V 15409
car_name_Hyundai Aura 15409
car_name_Hyundai Creta 15409
car_name_Hyundai Elantra 15409
car_name_Hyundai Grand 15409
car_name_Hyundai Santro 15409
car_name_Hyundai Tucson 15409
car_name_Hyundai Venue 15409
car_name_Hyundai Verna 15409
car_name_Hyundai i10 15409
car_name_Hyundai i20 15409
car_name_ISUZU MUX 15409
car_name_Isuzu D-Max 15409
car_name_Isuzu MUX 15409
car_name_Jaguar F-PACE 15409
car_name_Jaguar XE 15409
car_name_Jaguar XF 15409
car_name_Jeep Compass 15409
car_name_Jeep Wrangler 15409
car_name_Kia Carnival 15409
car_name_Kia Seltos 15409
car_name_Land Rover Rover 15409
car_name_Lexus ES 15409
car_name_Lexus NX 15409

car_name_Lexus RX 15409
car_name_MG Hector 15409
car_name_Mahindra Alturas 15409
car_name_Mahindra Bolero 15409
car_name_Mahindra KUV 15409
car_name_Mahindra KUV100 15409
car_name_Mahindra Marazzo 15409
car_name_Mahindra Scorpio 15409
car_name_Mahindra Thar 15409
car_name_Mahindra XUV300 15409
car_name_Mahindra XUV500 15409
car_name_Maruti Alto 15409
car_name_Maruti Baleno 15409
car_name_Maruti Celerio 15409
car_name_Maruti Ciaz 15409
car_name_Maruti Dzire LXI 15409
car_name_Maruti Dzire VXI 15409
car_name_Maruti Dzire ZXI 15409
car_name_Maruti Eeco 15409
car_name_Maruti Ertiga 15409
car_name_Maruti Ignis 15409
car_name_Maruti S-Presso 15409
car_name_Maruti Swift 15409
car_name_Maruti Swift Dzire 15409
car_name_Maruti Vitara 15409
car_name_Maruti Wagon R 15409
car_name_Maruti XL6 15409
car_name_Maserati Ghibli 15409
car_name_Maserati Quattroporte 15409
car_name_Mercedes-AMG C 15409
car_name_Mercedes-Benz C-Class 15409
car_name_Mercedes-Benz CLS 15409
car_name_Mercedes-Benz E-Class 15409
car_name_Mercedes-Benz GL-Class 15409
car_name_Mercedes-Benz GLS 15409
car_name_Mercedes-Benz S-Class 15409
car_name_Mini Cooper 15409
car_name_Nissan Kicks 15409
car_name_Nissan X-Trail 15409
car_name_Porsche Cayenne 15409
car_name_Porsche Macan 15409
car_name_Porsche Panamera 15409
car_name_Renault Duster 15409
car_name_Renault KWID 15409
car_name_Renault Triber 15409
car_name_Rolls-Royce Ghost 15409
car_name_Skoda Octavia 15409
car_name_Skoda Rapid 15409

car_name_Skoda Superb 15409
car_name_Tata Altroz 15409
car_name_Tata Harrier 15409
car_name_Tata Hexa 15409
car_name_Tata Nexon 15409
car_name_Tata Safari 15409
car_name_Tata Tiago 15409
car_name_Tata Tigor 15409
car_name_Toyota Camry 15409
car_name_Toyota Fortuner 15409
car_name_Toyota Glanza 15409
car_name_Toyota Innova 15409
car_name_Toyota Yaris 15409
car_name_Volkswagen Polo 15409
car_name_Volkswagen Vento 15409
car_name_Volvo S90 15409
car_name_Volvo XC 15409
car_name_Volvo XC60 15409
car_name_Volvo XC90 15409
brand_BMW 15409
brand_Bentley 15409
brand_Datsun 15409
brand_Ferrari 15409
brand_Force 15409
brand_Ford 15409
brand_Honda 15409
brand_Hyundai 15409
brand_ISUZU 15409
brand_Isuzu 15409
brand_Jaguar 15409
brand_Jeep 15409
brand_Kia 15409
brand_Land Rover 15409
brand_Lexus 15409
brand_MG 15409
brand_Mahindra 15409
brand_Maruti 15409
brand_Maserati 15409
brand_Mercedes-AMG 15409
brand_Mercedes-Benz 15409
brand_Mini 15409
brand_Nissan 15409
brand_Porsche 15409
brand_Renault 15409
brand_Rolls-Royce 15409
brand_Skoda 15409
brand_Tata 15409
brand_Toyota 15409

brand_Volkswagen 15409
brand_Volvo 15409
model_5 15409
model_6 15409
model_7 15409
model_A4 15409
model_A6 15409
model_A8 15409
model_Alto 15409
model_Altroz 15409
model_Alturas 15409
model_Amaze 15409
model_Aspire 15409
model_Aura 15409
model_Baleno 15409
model_Bolero 15409
model_C 15409
model_C-Class 15409
model_CLS 15409
model_CR 15409
model_CR-V 15409
model_Camry 15409
model_Carnival 15409
model_Cayenne 15409
model_Celerio 15409
model_Ciaz 15409
model_City 15409
model_Civic 15409
model_Compass 15409
model_Continental 15409
model_Cooper 15409
model_Creta 15409
model_D-Max 15409
model_Duster 15409
model_Dzire LXI 15409
model_Dzire VXI 15409
model_Dzire ZXI 15409
model_E-Class 15409
model_ES 15409
model_Ecosport 15409
model_Eeco 15409
model_Elantra 15409
model_Endeavour 15409
model_Ertiga 15409
model_F-PACE 15409
model_Figo 15409
model_Fortuner 15409
model_Freestyle 15409

model_GL-Class 15409
model_GLS 15409
model_GO 15409
model_GTC4Lusso 15409
model_Ghibli 15409
model_Ghost 15409
model_Glanza 15409
model_Grand 15409
model_Gurkha 15409
model_Harrier 15409
model_Hector 15409
model_Hexa 15409
model_Ignis 15409
model_Innova 15409
model_Jazz 15409
model_KUV 15409
model_KUV100 15409
model_KWID 15409
model_Kicks 15409
model_MUX 15409
model_Macan 15409
model_Marazzo 15409
model_NX 15409
model_Nexon 15409
model_Octavia 15409
model_Panamera 15409
model_Polo 15409
model_Q7 15409
model_Quattroporte 15409
model_RX 15409
model_Rapid 15409
model_RediGO 15409
model_Rover 15409
model_S-Class 15409
model_S-Presso 15409
model_S90 15409
model_Safari 15409
model_Santro 15409
model_Scorpio 15409
model_Seltos 15409
model_Superb 15409
model_Swift 15409
model_Swift Dzire 15409
model_Thar 15409
model_Tiago 15409
model_Tigor 15409
model_Triber 15409
model_Tucson 15409

```

model_Vento 15409
model_Venue 15409
model_Verna 15409
model_Vitara 15409
model_WR-V 15409
model_Wagon R 15409
model_Wrangler 15409
model_X-Trail 15409
model_X1 15409
model_X3 15409
model_X4 15409
model_X5 15409
model_XC 15409
model_XC60 15409
model_XC90 15409
model_XE 15409
model_XF 15409
model_XL6 15409
model_XUV300 15409
model_XUV500 15409
model_Yaris 15409
model_Z4 15409
model_i10 15409
model_i20 15409
model_redi-GO 15409
seller_type_Individual 15409
seller_type_Trustmark Dealer 15409
fuel_type_Diesel 15409
fuel_type_Electric 15409
fuel_type_LPG 15409
fuel_type_Petrol 15409
transmission_type_Manual 15409

```

```
[29]: df2=df2.dropna()
```

```
[30]: df2.isnull().sum()
```

```

[30]: city          0
      make          0
      model         0
      variant       0
      mileage       0
      make_year     0
      price         0
      fuel_type     0
      no_of_owners  0
      color         0

```

```
body_type          0
transmission       0
registration_year  0
latest_publish_date 0
dtype: int64
```

```
[31]: df2.columns
```

```
[31]: Index(['city', 'make', 'model', 'variant', 'mileage', 'make_year', 'price',
          'fuel_type', 'no_of_owners', 'color', 'body_type', 'transmission',
          'registration_year', 'latest_publish_date'],
          dtype='str')
```

```
[32]: df2["registration_year"]
```

```
[32]: 0      2021
      1      2021
      2      2018
      3      2018
      4      2017
      ...
     8132    2018
     8133    2021
     8134    2022
     8135    2016
     8136    2021
      Name: registration_year, Length: 8136, dtype: int64
```

CHECKING SKEWNESS IN DATA USING PROBPLOT/QQPLOT

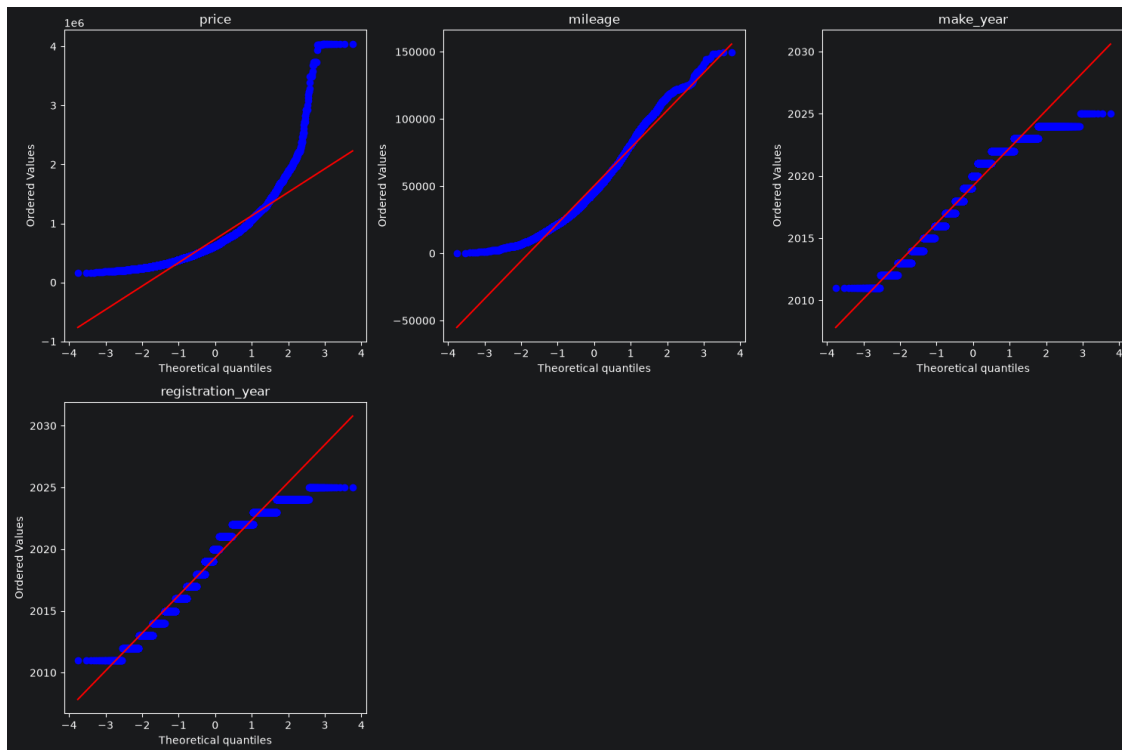
```
[33]: from scipy import stats
import matplotlib.pyplot as plt

columns = [
    "price",
    "mileage",
    "make_year",
    "registration_year",
]

plt.figure(figsize=(15,10))

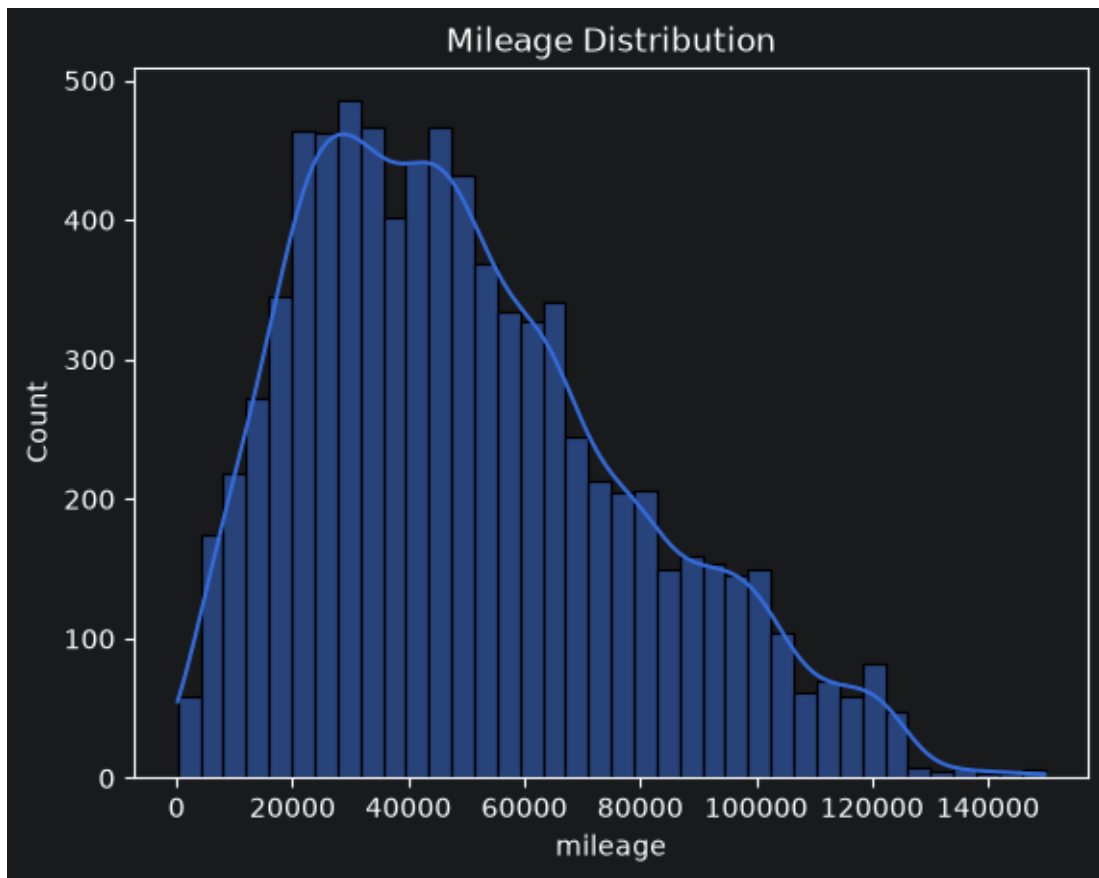
for i, col in enumerate(columns, 1):
    plt.subplot(2, 3, i)    # 2 rows, 3 columns
    stats.probplot(df2[col], dist="norm", plot=plt)
    plt.title(col)
```

```
plt.tight_layout()
plt.show()
```



```
[34]: import seaborn as sns
import matplotlib.pyplot as plt

sns.histplot(df2['mileage'], kde=True)
plt.title('Mileage Distribution')
plt.show()
```



NOW TRAINING A MODEL ON DELHI CAR DATASET WITHOUT ANY TRANSFORMATIONS

```
[35]: print(df2["price"].dtype)
```

float64

Checking datatypes of all columns

```
[36]: print(df2.dtypes)
```

city	str
make	str
model	str
variant	str
mileage	int64
make_year	int64
price	float64
fuel_type	str
no_of_owners	int64
color	str
body_type	str
transmission	str

```

registration_year      int64
latest_publish_date     str
dtype: object

```

```
[37]: df.head()
```

```

[37]: Unnamed: 0  vehicle_age  km_driven  mileage  engine  max_power  seats  \
0           0           9    120000    19.70     796      46.30     5
1           1           5     20000    18.90    1197      82.00     5
2           2          11     60000    17.00    1197      80.00     5
3           3           9     37000    20.92     998      67.10     5
4           4           6     30000    22.77    1498      98.59     5

```

```

      selling_price  car_name_Audi A6  car_name_Audi A8  ...  model_i10  \
0         120000          False          False  ...      False
1         550000          False          False  ...      False
2         215000          False          False  ...      False
3         226000          False          False  ...      False
4         570000          False          False  ...      False

```

```

      model_i20  model_redi-GO  seller_type_Individual  \
0         False          False          True
1         False          False          True
2          True          False          True
3         False          False          True
4         False          False          False

```

```

      seller_type_Trustmark Dealer  fuel_type_Diesel  fuel_type_Electric  \
0              False          False          False          False
1              False          False          False          False
2              False          False          False          False
3              False          False          False          False
4              False          True          False          False

```

```

      fuel_type_LPG  fuel_type_Petrol  transmission_type_Manual
0         False          True          True
1         False          True          True
2         False          True          True
3         False          True          True
4         False          False          True

```

```
[5 rows x 285 columns]
```

```

[51]: import pandas as pd
      from sklearn.preprocessing import LabelEncoder
      le=LabelEncoder()
      df2=pd.read_csv("delhicar.csv")
      df2=df2.drop("latest_publish_date",axis=1)

```

```

df2=pd.get_dummies(df2,drop_first=True)
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score
x=df2.drop("price",axis=1)
y=df2[["price"]]
model=LinearRegression()
X_train,X_test,y_train,y_test=train_test_split(x,y,test_size=0.
↪2,random_state=42)
model.fit(X_train,y_train)
prediction=model.predict(X_test)
r2score=r2_score(y_test,prediction)

train_pred = model.predict(X_train)
test_pred = model.predict(X_test)

print("Train R²:", r2_score(y_train, train_pred))
print("Test R²:", r2_score(y_test, test_pred))

print(r2score)
df.head()

```

Train R²: 0.9535307574015496

Test R²: 0.9493015254768048

0.9493015254768048

```

[51]: Unnamed: 0  vehicle_age  km_driven  mileage  engine  max_power  seats  \
0            0           9    120000    19.70    796      46.30    5
1            1           5     20000    18.90   1197      82.00    5
2            2          11     60000    17.00   1197      80.00    5
3            3           9     37000    20.92    998      67.10    5
4            4           6     30000    22.77   1498      98.59    5

    selling_price  car_name_Audi A6  car_name_Audi A8  ...  model_i10  \
0         120000          False          False  ...    False
1         550000          False          False  ...    False
2         215000          False          False  ...    False
3         226000          False          False  ...    False
4         570000          False          False  ...    False

    model_i20  model_redi-GO  seller_type_Individual  \
0         False          False                True
1         False          False                True
2          True          False                True
3         False          False                True
4         False          False                False

```


	seller_type_Trustmark Dealer	fuel_type_Diesel	fuel_type_Electric \
0	False	False	False
1	False	False	False
2	False	False	False
3	False	False	False
4	False	True	False

	fuel_type_LPG	fuel_type_Petrol	transmission_type_Manual
0	False	True	True
1	False	True	True
2	False	True	True
3	False	True	True
4	False	False	True

[5 rows x 285 columns]

After training linear regression model we can see that we are getting score of 0.95 but there might be chance that model may overfit so we need to check if onehot encoding got applied in wrong column which contains many unique values and total rows/unique value is <10 because this will create multiple columns and model will overfit or byheart those columns

FIRST TRYING TO SEE WHAT WILL HAPPEN IF WE REMOVE VARIANTS COLUMN BECAUSE IT HAS MANY UNIQUE VALUES

```
[2]: import pandas as pd
from sklearn.preprocessing import LabelEncoder
le=LabelEncoder()
df2=pd.read_csv("delhicar.csv")
df2=df2.drop("latest_publish_date",axis=1)
df2=df2.drop("variant",axis=1)
df2=pd.get_dummies(df2,drop_first=True)
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score
x=df2.drop("price",axis=1)
y=df2[["price"]]
model=LinearRegression()
X_train,X_test,y_train,y_test=train_test_split(x,y,test_size=0.
↪2,random_state=42)
model.fit(X_train,y_train)
prediction=model.predict(X_test)
r2score=r2_score(y_test,prediction)

train_pred = model.predict(X_train)
test_pred = model.predict(X_test)
```

```
print("Train R²:", r2_score(y_train, train_pred))
print("Test R²:", r2_score(y_test, test_pred))

print(r2score)
```

```
Train R²: 0.9018184598189581
Test R²: 0.8657624317308963
0.8657624317308963
```

After removing variants column now we can see that we got r square of 0.86 which includes that variant is an important feature but we should solve curse of dimensionality for that particular column we could now do that by grouping rare brands under rare_brands column so that number of unique variants gets reduced because if we don't increase total rows/unique to >10 it will create multicollinearity and model will overfit

```
[ ]: df2=df2.drop("latest_publish_date",axis=1)
```

```
[39]: pd.read_csv("delhicar.csv").nunique().sort_values(ascending=False).head(20)
```

```
[39]: latest_publish_date    5658
      mileage              5604
      price                1963
      variant              1343
      model                158
      make                 21
      color                16
      city                 16
      make_year            15
      registration_year    15
      body_type            6
      fuel_type            6
      no_of_owners         3
      transmission        2
      dtype: int64
```

```
[40]: raw_df = pd.read_csv("delhicar.csv")
      print(raw_df.dtypes)
```

```
city          str
make          str
model         str
variant       str
mileage       int64
make_year     int64
price         float64
fuel_type     str
no_of_owners  int64
color         str
body_type     str
```

```

transmission          str
registration_year      int64
latest_publish_date    str
dtype: object

```

```
[43]: df2.columns
```

```
[43]: Index(['mileage', 'make_year', 'price', 'no_of_owners', 'registration_year',
            'city_bangalore', 'city_chandigarh', 'city_chennai', 'city_delhi',
            'city_delhi-ncr',
            ...,
            'color_silver', 'color_violet', 'color_white', 'color_yellow',
            'body_type_hatchback', 'body_type_muv', 'body_type_sedan',
            'body_type_suv', 'body_type_wagon', 'transmission_manual'],
            dtype='str', length=1565)
```

```
[5]: df2=pd.read_csv("delhicar.csv")
df2["variant"].value_counts()
df2.nunique()
```

```
[5]: city          16
make             21
model           158
variant         1343
mileage         5604
make_year        15
price           1963
fuel_type        6
no_of_owners     3
color            16
body_type        6
transmission     2
registration_year 15
latest_publish_date 5658
dtype: int64
```

```
[6]: counts = df2["variant"].value_counts()
print(counts[counts<15])
print("Variants with <5 rows:", (counts < 5).sum())
print("Variants with <10 rows:", (counts < 10).sum())
print("Variants with <15 rows:", (counts < 15).sum())
print("Variants with <20 rows:", (counts < 20).sum())
```

```

variant
Alpha 1.5 AT          14
Delta 1.4 MT          14
XT Petrol             14
HTX G                 14

```

```

1.6 SX AT VTVT                                14
..
Comfortline Plus 1.0L MPI                      1
Trendline 1.0 Petrol+cng(outside fitted)       1
4x2 AT                                           1
1.5 S MT Diesel                                1
SX (0) Connect 1.2 MT                          1
Name: count, Length: 1217, dtype: int64
Variants with <5 rows: 907
Variants with <10 rows: 1126
Variants with <15 rows: 1217
Variants with <20 rows: 1267

```

```
[9]: print((counts<5).sum())
```

```
907
```

Now to remove curse of dimensionality we should group rare brands whose count is less than 15 into rare category and then apply one-hot encoding

```
[12]: import pandas as pd
from sklearn.preprocessing import LabelEncoder
le=LabelEncoder()
df2=pd.read_csv("delhicar.csv")

# remove curse of dimensionality from variants by grouping rare variants under
↳ other
valuecounts=df2["variant"].value_counts()
rarelements=valuecounts[valuecounts<15].index
df2["variant"]=df2["variant"].replace(rarelements,"Other")
df2=df2.drop("latest_publish_date",axis=1)
df2=pd.get_dummies(df2,drop_first=True)
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score
x=df2.drop("price",axis=1)
y=df2[["price"]]
model=LinearRegression()
X_train,X_test,y_train,y_test=train_test_split(x,y,test_size=0.
↳ 2,random_state=42)
model.fit(X_train,y_train)
prediction=model.predict(X_test)
r2score=r2_score(y_test,prediction)

train_pred = model.predict(X_train)
test_pred = model.predict(X_test)
```

```

print("Train R2:", r2_score(y_train, train_pred))
print("Test R2:", r2_score(y_test, test_pred))

print(r2score)

```

Train R²: 0.9080644739692731

Test R²: 0.870255142259133

0.870255142259133

Now applying ONe-hot Encoding on price from above code

```

[13]: import pandas as pd
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import r2_score, mean_absolute_error
import numpy as np

le = LabelEncoder()
df2 = pd.read_csv("delhicar.csv")

# remove curse of dimensionality from variants by grouping rare variants under
↳ "Other"
valuecounts = df2["variant"].value_counts()
rarelements = valuecounts[valuecounts < 15].index
df2["variant"] = df2["variant"].replace(rarelements, "Other")

print("Variant categories after grouping:", df2["variant"].nunique()) # ↵
↳ INSERT HERE

df2 = df2.drop("latest_publish_date", axis=1)
df2 = pd.get_dummies(df2, drop_first=True) # variant becomes variant_XXX
↳ columns from here onward

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score
df2["price"] = np.log1p(df2["price"])
x = df2.drop("price", axis=1)
y = df2["price"]

model = LinearRegression()
X_train, X_test, y_train, y_test = train_test_split(x, y, test_size=0.2, ↵
↳ random_state=42)
model.fit(X_train, y_train)

train_pred = model.predict(X_train)
test_pred = model.predict(X_test)
print("Train R2:", r2_score(y_train, train_pred))
print("Test R2:", r2_score(y_test, test_pred))

```

Variant categories after grouping: 127

Train R^2 : 0.969384118369973

Test R^2 : 0.9606493575585063

AFTER APPLYING LOG TRANSFORM ON PRICE THE R SQAURE JUMPED FROM 0.89 TO 0.9604

Conclusion :-the max rsquare which we got from liner regression on cardexho dataset and delhicar dataset where 0.89 and 0.96 respectively we ar egetitng more score in dekhicar dataset because it ha sother imortnat features such as color ,owner and many othr factors that play an important role in a car's selling price.

Next steps :- we will contintue workign o car dekho dataset because it its data isnt limited to any state and no further optimizations on model trained on delhicar dataset because it has rsquare sore of 0.96 already.

First we have to design a pipeline for Linear Regression Model each model will be having its own pipeline

```
[2]: import pandas as pd
import numpy as np
from sklearn.preprocessing import
    ↳StandardScaler,OneHotEncoder,FunctionTransformer
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_absolute_error
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
linearmodeldf = pd.read_csv("car.csv")
print(linearmodeldf.columns.tolist())

X = linearmodeldf.drop("selling_price", axis=1)
y = linearmodeldf["selling_price"]
# log transform target
y_log = np.log1p(y)
X_train, X_test, y_train_log, y_test_log = train_test_split(
X, y_log, test_size=0.2, random_state=42
)
tf1=ColumnTransformer([
    ↳
    ↳("Onehotencoder",OneHotEncoder(sparse_output=False,handle_unknown="ignore"),["car_name","br
        ("Logtransformer",FunctionTransformer(np.log1p),["km_driven"]),
    ↳
    ↳("StandardScaler",StandardScaler(),["vehicle_age","engine","max_power","mileage"]),
])
pipeline=Pipeline(
    [
```

```

        ("preprocessor",tf1),
        (
            "model",LinearRegression()
        )
    ]
)

pipeline.fit(X_train, y_train_log)
# predict log price

# testing model with custom input

# custom_input = pd.DataFrame({
#     "car_name": ["Honda City"],
#     "brand": ["Honda"],
#     "model": ["City"],
#     "vehicle_age": [5],
#     "km_driven": [45000],
#     "seller_type": ["Dealer"],
#     "fuel_type": ["Petrol"],
#     "engine": [1498],
#     "mileage": [17.8],
#     "max_power": [117.3]
# })

custom_input = pd.DataFrame({
    "car_name": ["Mercedes-Benz C-Class"],
    "brand": ["Mercedes-Benz"],
    "model": ["C-Class"],
    "vehicle_age": [4],
    "km_driven": [38000],
    "seller_type": ["Dealer"],
    "fuel_type": ["Diesel"],
    "engine": [1950],
    "mileage": [19.0],
    "max_power": [191.0]
})

pred_log = pipeline.predict(custom_input)
pred_price = np.expm1(pred_log)

print(f"Predicted Price: {pred_price[0]:,.0f}")

pred_log = pipeline.predict(X_test)
# convert back to actual price
pred_price = np.expm1(pred_log)

```

```

y_test_price = np.expml(y_test_log)
print("R2 Score:", round(r2_score(y_test_price, pred_price), 4))
print("MAE:", round(mean_absolute_error(y_test_price, pred_price), 2))

```

```

['Unnamed: 0', 'car_name', 'brand', 'model', 'vehicle_age', 'km_driven',
'seller_type', 'fuel_type', 'transmission_type', 'mileage', 'engine',
'max_power', 'seats', 'selling_price']
Predicted Price: 2,438,945
R2 Score: 0.9061
MAE: 118263.41

```

```

[17]: import pandas as pd
testdf=pd.read_csv("car.csv")
testdf.nunique()

```

```

[17]: Unnamed: 0      15411
car_name           121
brand              32
model             120
vehicle_age        24
km_driven          3688
seller_type         3
fuel_type           5
transmission_type   2
mileage            411
engine             110
max_power           342
seats               8
selling_price      1086
dtype: int64

```

Achieved max r square of 0.907 in linear regression model on cardkho dataset and Created a compelte linear regression pipeline

```

[3]: print(linearmodeldf.shape[0] / linearmodeldf['car_name'].nunique())

```

```

127.36363636363636

```

Exporting LinearRegression Pipeline

```

[3]: import pandas as pd
import numpy as np
import joblib
from sklearn.preprocessing import
    StandardScaler, OneHotEncoder, FunctionTransformer
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_absolute_error
from sklearn.pipeline import Pipeline

```



```

from sklearn.compose import ColumnTransformer
linearmodeldf = pd.read_csv("car.csv")
print(linearmodeldf.columns.tolist())

X = linearmodeldf.drop("selling_price", axis=1)
y = linearmodeldf["selling_price"]
# log transform target transmission_type
y_log = np.log1p(y)
X_train, X_test, y_train_log, y_test_log = train_test_split(
X, y_log, test_size=0.2, random_state=42
)
tf1=ColumnTransformer([
    ↳
    ↳("Onehotencoder",OneHotEncoder(sparse_output=False,handle_unknown="ignore"),["car_name","br
    ↳("Logtransformer",FunctionTransformer(np.log1p),["km_driven"])),
    ↳
    ↳("StandardScaler",StandardScaler(),["vehicle_age","engine","max_power","mileage"])),
])
pipeline=Pipeline(
    [
        ("preprocessor",tf1),
        (
            "model",LinearRegression()
        )
    ]
)

pipeline.fit(X_train, y_train_log)
# predict log price

# testing model with custom input

# custom_input = pd.DataFrame({
#     "car_name": ["Honda City"],
#     "brand": ["Honda"],
#     "model": ["City"],
#     "vehicle_age": [5],
#     "km_driven": [45000],
#     "seller_type": ["Dealer"],
#     "fuel_type": ["Petrol"],
#     "engine": [1498],
#     "mileage": [17.8],
#     "max_power": [117.3]
# })

```

```

custom_input = pd.DataFrame({
    "car_name": ["Mercedes-Benz C-Class"],
    "brand": ["Mercedes-Benz"],
    "model": ["C-Class"],
    "vehicle_age": [4],
    "km_driven": [38000],
    "seller_type": ["Dealer"],
    "fuel_type": ["Diesel"],
    "engine": [1950],
    "mileage": [19.0],
    "max_power": [191.0],
})

# calculating train r square
train_pred=pipeline.predict(X_train)

pred_log = pipeline.predict(custom_input)
pred_price = np.expml(pred_log)

print(f"Predicted Price: {pred_price[0]:,.0f}")

pred_log = pipeline.predict(X_test)
# convert back to actual price
pred_price = np.expml(pred_log)
y_test_price = np.expml(y_test_log)
print("R2 test Score:", round(r2_score(y_test_price, pred_price), 4))
print("r2 train score", r2_score(np.expml(train_pred), np.expml(y_train_log)))
print("MAE:", round(mean_absolute_error(y_test_price, pred_price), 2))

# joblib.dump(pipeline, "linearmodel.pkl")

```

```

['Unnamed: 0', 'car_name', 'brand', 'model', 'vehicle_age', 'km_driven',
'seller_type', 'fuel_type', 'transmission_type', 'mileage', 'engine',
'max_power', 'seats', 'selling_price']
Predicted Price: 2,438,945
R2 test Score: 0.9061
r2 train score 0.9157502772438825
MAE: 118263.41

```

Now traing Rando forest

```

[6]: import pandas as pd
import numpy as np
import joblib
from sklearn.preprocessing import
    ↳StandardScaler, OneHotEncoder, FunctionTransformer

```

```

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_absolute_error
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.ensemble import RandomForestRegressor
linearmodeldf = pd.read_csv("car.csv")
print(linearmodeldf.columns.tolist())

X = linearmodeldf.drop("selling_price", axis=1)
y = linearmodeldf["selling_price"]
# log transform target
y_log = np.log1p(y)
X_train, X_test, y_train_log, y_test_log = train_test_split(
X, y_log, test_size=0.2, random_state=42
)
tf1=ColumnTransformer([
    ↳
    ↳("Onehotencoder",OneHotEncoder(sparse_output=False,handle_unknown="ignore"),["car_name","br
        ("Logtransformer",FunctionTransformer(np.log1p),["km_driven"]),
    ↳
    ↳("StandardScaler",StandardScaler(),["vehicle_age","engine","max_power","mileage"]),
])
pipeline=Pipeline(
    [
        ("preprocessor",tf1),
        (
            "model",RandomForestRegressor()
        )
    ]
)

pipeline.fit(X_train, y_train_log)
# predict log price

# testing model with custom input

# custom_input = pd.DataFrame({
#     "car_name": ["Honda City"],
#     "brand": ["Honda"],
#     "model": ["City"],
#     "vehicle_age": [5],
#     "km_driven": [45000],
#     "seller_type": ["Dealer"],

```

```

#     "fuel_type": ["Petrol"],
#     "engine": [1498],
#     "mileage": [17.8],
#     "max_power": [117.3]
# })

custom_input = pd.DataFrame({
    "car_name": ["Mercedes-Benz C-Class"],
    "brand": ["Mercedes-Benz"],
    "model": ["C-Class"],
    "vehicle_age": [4],
    "km_driven": [38000],
    "seller_type": ["Dealer"],
    "fuel_type": ["Diesel"],
    "engine": [1950],
    "mileage": [19.0],
    "max_power": [191.0]
})
# calculating train r square
train_pred=pipeline.predict(X_train)

pred_log = pipeline.predict(custom_input)
pred_price = np.expml(pred_log)

print(f"Predicted Price: {pred_price[0]:,.0f}")

pred_log = pipeline.predict(X_test)
# convert back to actual price
pred_price = np.expml(pred_log)
y_test_price = np.expml(y_test_log)
print("R2 test Score:", round(r2_score(y_test_price, pred_price), 4))
print("r2 train score", r2_score(np.expml(train_pred), np.expml(y_train_log)))
print("MAE:", round(mean_absolute_error(y_test_price, pred_price), 2))

joblib.dump(pipeline, "randommodel.pkl")

```

```

['Unnamed: 0', 'car_name', 'brand', 'model', 'vehicle_age', 'km_driven',
'seller_type', 'fuel_type', 'transmission_type', 'mileage', 'engine',
'max_power', 'seats', 'selling_price']

```

Predicted Price: 3,493,157

R2 test Score: 0.9318

r2 train score 0.9610025160468607

MAE: 100870.98

KeyboardInterrupt

Traceback (most recent call last)

```

Cell In[6], line 85
    81 print("R2 test Score:", round(r2_score(y_test_price, pred_price), 4))
    82 print("r2 train score", r2_score(np.expm1(train_pred), np.
↳ expm1(y_train_log)))
    83 print("MAE:", round(mean_absolute_error(y_test_price, pred_price), 2))
    84
---> 85 joblib.dump(pipeline, "randommodel.pkl")

```

```

File ~/PycharmProjects/AUTOWORTH_BACKEND/.venv/lib/python3.14/site-packages/
↳ joblib/numpy_pickle.py:600, in dump(value, filename, compress, protocol)
    598 elif is_filename:
    599     with open(filename, "wb") as f:
--> 600         NumpyPickler(f, protocol=protocol).dump(value)
    601 else:
    602     NumpyPickler(filename, protocol=protocol).dump(value)

```

```

File /usr/lib/python3.14/pickle.py:498, in _Pickler.dump(self, obj)
    496 if self.proto >= 4:
    497     self.framer.start_framing()
--> 498 self.save(obj)
    499 self.write(STOP)
    500 self.framer.end_framing()

```

```

File ~/PycharmProjects/AUTOWORTH_BACKEND/.venv/lib/python3.14/site-packages/
↳ joblib/numpy_pickle.py:395, in NumpyPickler.save(self, obj)
    392 wrapper.write_array(obj, self)
    393 return
--> 395 return Pickler.save(self, obj)

```

```

File /usr/lib/python3.14/pickle.py:615, in _Pickler.save(self, obj,
↳ save_persistent_id)
    611 raise PicklingError("tuple returned by __reduce__ "
    612                      "must contain 2 through 6 elements")
    614 # Save the reduce() output and finally memoize the object
--> 615 self.save_reduce(obj=obj, *rv)
    616 except BaseException as exc:
    617     exc.add_note(f'when serializing {_T(obj)} object')

```

```

File /usr/lib/python3.14/pickle.py:761, in _Pickler.save_reduce(self, func,
↳ args, state, listitems, dictitems, state_setter, obj)
    759 if state_setter is None:
    760     try:
--> 761         save(state)
    762     except BaseException as exc:
    763         exc.add_note(f'when serializing {_T(obj)} state')

```

```

File ~/PycharmProjects/AUTOWORTH_BACKEND/.venv/lib/python3.14/site-packages/
↳ joblib/numpy_pickle.py:395, in NumpyPickler.save(self, obj)

```

```

    392     wrapper.write_array(obj, self)
    393     return
--> 395 return Pickler.save(self, obj)

```

File /usr/lib/python3.14/pickle.py:572, in _Pickler.save(self, obj,
↪save_persistent_id)

```

    570 f = self.dispatch.get(t)
    571 if f is not None:
--> 572     f(self, obj) # Call unbound method with explicit self
    573     return
    575 # Check private dispatch table if any, or else
    576 # copyreg.dispatch_table

```

File /usr/lib/python3.14/pickle.py:1058, in _Pickler.save_dict(self, obj)

```

    1055     self.write(MARK + DICT)
    1057 self.memoize(obj)
-> 1058 self._batch_setitems(obj.items(), obj)

```

File /usr/lib/python3.14/pickle.py:1084, in _Pickler._batch_setitems(self,
↪items, obj)

```

    1082 save(k)
    1083 try:
-> 1084     save(v)
    1085 except BaseException as exc:
    1086     exc.add_note(f'when serializing {_T(obj)} item {k!r}')

```

File ~/PycharmProjects/AUTOWORTH_BACKEND/.venv/lib/python3.14/site-packages/

↪joblib/numpy_pickle.py:395, in NumpyPickler.save(self, obj)

```

    392     wrapper.write_array(obj, self)
    393     return
--> 395 return Pickler.save(self, obj)

```

File /usr/lib/python3.14/pickle.py:572, in _Pickler.save(self, obj,
↪save_persistent_id)

```

    570 f = self.dispatch.get(t)
    571 if f is not None:
--> 572     f(self, obj) # Call unbound method with explicit self
    573     return
    575 # Check private dispatch table if any, or else
    576 # copyreg.dispatch_table

```

File /usr/lib/python3.14/pickle.py:1009, in _Pickler.save_list(self, obj)

```

    1006     self.write(MARK + LIST)
    1008 self.memoize(obj)
-> 1009 self._batch_appends(obj, obj)

```

File /usr/lib/python3.14/pickle.py:1037, in _Pickler._batch_appends(self, items
↪obj)

```

1035 for i, x in enumerate(batch, start):
1036     try:
-> 1037         save(x)
1038     except BaseException as exc:
1039         exc.add_note(f'when serializing {_T(obj)} item {i}')
```

File ~/PycharmProjects/AUTOWORTH_BACKEND/.venv/lib/python3.14/site-packages/joblib/numpy_pickle.py:395, in NumpyPickler.save(self, obj)

```

392     wrapper.write_array(obj, self)
393     return
--> 395 return Pickler.save(self, obj)
```

File /usr/lib/python3.14/pickle.py:572, in _Pickler.save(self, obj,
↪save_persistent_id)

```

570 f = self.dispatch.get(t)
571 if f is not None:
--> 572     f(self, obj) # Call unbound method with explicit self
573     return
575 # Check private dispatch table if any, or else
576 # copyreg.dispatch_table
```

File /usr/lib/python3.14/pickle.py:957, in _Pickler.save_tuple(self, obj)

```

955 for i, element in enumerate(obj):
956     try:
--> 957         save(element)
958     except BaseException as exc:
959         exc.add_note(f'when serializing {_T(obj)} item {i}')
```

File ~/PycharmProjects/AUTOWORTH_BACKEND/.venv/lib/python3.14/site-packages/joblib/numpy_pickle.py:395, in NumpyPickler.save(self, obj)

```

392     wrapper.write_array(obj, self)
393     return
--> 395 return Pickler.save(self, obj)
```

File /usr/lib/python3.14/pickle.py:615, in _Pickler.save(self, obj,
↪save_persistent_id)

```

611     raise PicklingError("tuple returned by __reduce__ "
612                          "must contain 2 through 6 elements")
614     # Save the reduce() output and finally memoize the object
--> 615     self.save_reduce(obj=obj, *rv)
616 except BaseException as exc:
617     exc.add_note(f'when serializing {_T(obj)} object')
```

File /usr/lib/python3.14/pickle.py:761, in _Pickler.save_reduce(self, func,
↪args, state, listitems, dictitems, state_setter, obj)

```

759 if state_setter is None:
760     try:
--> 761         save(state)
```

```

762     except BaseException as exc:
763         exc.add_note(f'when serializing {_T(obj)} state')

File ~/PycharmProjects/AUTOWORTH_BACKEND/.venv/lib/python3.14/site-packages/
↳joblib/numpy_pickle.py:395, in NumpyPickler.save(self, obj)
    392     wrapper.write_array(obj, self)
    393     return
--> 395 return Pickler.save(self, obj)

File /usr/lib/python3.14/pickle.py:572, in _Pickler.save(self, obj,
↳save_persistent_id)
    570 f = self.dispatch.get(t)
    571 if f is not None:
--> 572     f(self, obj) # Call unbound method with explicit self
    573     return
    575 # Check private dispatch table if any, or else
    576 # copyreg.dispatch_table

File /usr/lib/python3.14/pickle.py:1058, in _Pickler.save_dict(self, obj)
    1055     self.write(MARK + DICT)
    1057 self.memoize(obj)
-> 1058 self._batch_setitems(obj.items(), obj)

File /usr/lib/python3.14/pickle.py:1084, in _Pickler._batch_setitems(self,
↳items, obj)
    1082 save(k)
    1083 try:
-> 1084     save(v)
    1085 except BaseException as exc:
    1086     exc.add_note(f'when serializing {_T(obj)} item {k!r}')

File ~/PycharmProjects/AUTOWORTH_BACKEND/.venv/lib/python3.14/site-packages/
↳joblib/numpy_pickle.py:395, in NumpyPickler.save(self, obj)
    392     wrapper.write_array(obj, self)
    393     return
--> 395 return Pickler.save(self, obj)

File /usr/lib/python3.14/pickle.py:572, in _Pickler.save(self, obj,
↳save_persistent_id)
    570 f = self.dispatch.get(t)
    571 if f is not None:
--> 572     f(self, obj) # Call unbound method with explicit self
    573     return
    575 # Check private dispatch table if any, or else
    576 # copyreg.dispatch_table

File /usr/lib/python3.14/pickle.py:1009, in _Pickler.save_list(self, obj)
    1006     self.write(MARK + LIST)

```



```

1008 self.memoize(obj)
-> 1009 self._batch_appends(obj, obj)

File /usr/lib/python3.14/pickle.py:1037, in _Pickler._batch_appends(self, items,
↳obj)
    1035 for i, x in enumerate(batch, start):
    1036     try:
-> 1037         save(x)
    1038     except BaseException as exc:
    1039         exc.add_note(f'when serializing {_T(obj)} item {i}')
```

```

File ~/PycharmProjects/AUTOWORTH_BACKEND/.venv/lib/python3.14/site-packages/
↳joblib/numpy_pickle.py:395, in NumpyPickler.save(self, obj)
    392     wrapper.write_array(obj, self)
    393     return
--> 395 return Pickler.save(self, obj)

File /usr/lib/python3.14/pickle.py:615, in _Pickler.save(self, obj,
↳save_persistent_id)
    611     raise PicklingError("tuple returned by __reduce__ "
    612                          "must contain 2 through 6 elements")
    614     # Save the reduce() output and finally memoize the object
--> 615     self.save_reduce(obj=obj, *rv)
    616 except BaseException as exc:
    617     exc.add_note(f'when serializing {_T(obj)} object')
```

```

File /usr/lib/python3.14/pickle.py:761, in _Pickler.save_reduce(self, func,
↳args, state, listitems, dictitems, state_setter, obj)
    759 if state_setter is None:
    760     try:
--> 761         save(state)
    762     except BaseException as exc:
    763         exc.add_note(f'when serializing {_T(obj)} state')
```

```

File ~/PycharmProjects/AUTOWORTH_BACKEND/.venv/lib/python3.14/site-packages/
↳joblib/numpy_pickle.py:395, in NumpyPickler.save(self, obj)
    392     wrapper.write_array(obj, self)
    393     return
--> 395 return Pickler.save(self, obj)

File /usr/lib/python3.14/pickle.py:572, in _Pickler.save(self, obj,
↳save_persistent_id)
    570 f = self.dispatch.get(t)
    571 if f is not None:
--> 572     f(self, obj) # Call unbound method with explicit self
    573     return
    575 # Check private dispatch table if any, or else
    576 # copyreg.dispatch_table
```

```

File /usr/lib/python3.14/pickle.py:1058, in _Pickler.save_dict(self, obj)
    1055     self.write(MARK + DICT)
    1057     self.memoize(obj)
-> 1058     self._batch_setitems(obj.items(), obj)

File /usr/lib/python3.14/pickle.py:1084, in _Pickler._batch_setitems(self,
↪items, obj)
    1082     save(k)
    1083     try:
-> 1084         save(v)
    1085     except BaseException as exc:
    1086         exc.add_note(f'when serializing {_T(obj)} item {k!r}')

[... skipping similar frames: NumpyPickler.save at line 395 (1 times)]

File /usr/lib/python3.14/pickle.py:615, in _Pickler.save(self, obj,
↪save_persistent_id)
    611         raise PicklingError("tuple returned by __reduce__ "
    612                               "must contain 2 through 6 elements")
    614     # Save the reduce() output and finally memoize the object
--> 615     self.save_reduce(obj=obj, *rv)
    616     except BaseException as exc:
    617         exc.add_note(f'when serializing {_T(obj)} object')

File /usr/lib/python3.14/pickle.py:761, in _Pickler.save_reduce(self, func,
↪args, state, listitems, dictitems, state_setter, obj)
    759     if state_setter is None:
    760         try:
--> 761             save(state)
    762         except BaseException as exc:
    763             exc.add_note(f'when serializing {_T(obj)} state')

File ~/PycharmProjects/AUTOWORTH_BACKEND/.venv/lib/python3.14/site-packages/
↪joblib/numpy_pickle.py:395, in NumpyPickler.save(self, obj)
    392     wrapper.write_array(obj, self)
    393     return
--> 395     return Pickler.save(self, obj)

File /usr/lib/python3.14/pickle.py:572, in _Pickler.save(self, obj,
↪save_persistent_id)
    570     f = self.dispatch.get(t)
    571     if f is not None:
--> 572         f(self, obj) # Call unbound method with explicit self
    573     return
    575     # Check private dispatch table if any, or else
    576     # copyreg.dispatch_table

```

```
File /usr/lib/python3.14/pickle.py:1058, in _Pickler.save_dict(self, obj)
    1055     self.write(MARK + DICT)
    1057 self.memoize(obj)
-> 1058 self._batch_setitems(obj.items(), obj)
```

```
File /usr/lib/python3.14/pickle.py:1084, in _Pickler._batch_setitems(self,
↪ items, obj)
    1082 save(k)
    1083 try:
-> 1084     save(v)
    1085 except BaseException as exc:
    1086     exc.add_note(f'when serializing {_T(obj)} item {k!r}')

```

```
File ~/PycharmProjects/AUTOWORTH_BACKEND/.venv/lib/python3.14/site-packages/
↪ joblib/numpy_pickle.py:392, in NumpyPickler.save(self, obj)
    389     self.framer.commit_frame(force=True)
    391     # And then array bytes are written right after the wrapper.
--> 392     wrapper.write_array(obj, self)
    393     return
    395 return Pickler.save(self, obj)
```

```
File ~/PycharmProjects/AUTOWORTH_BACKEND/.venv/lib/python3.14/site-packages/
↪ joblib/numpy_pickle.py:157, in NumpyArrayWrapper.write_array(self, array,
↪ pickler)
    149     pickler.file_handle.write(padding)
    151 for chunk in pickler.np.nditer(
    152     array,
    153     flags=["external_loop", "buffered", "zerosize_ok"],
    154     buffersize=buffersize,
    155     order=self.order,
    156 ):
--> 157     pickler.file_handle.write(chunk.tobytes("C"))
```

KeyboardInterrupt:

```
when serializing dict item 'nodes'
when serializing sklearn.tree._tree.Tree state
when serializing sklearn.tree._tree.Tree object
when serializing dict item 'tree_'
when serializing sklearn.tree._classes.DecisionTreeRegressor state
when serializing sklearn.tree._classes.DecisionTreeRegressor object
when serializing list item 65
when serializing dict item 'estimators_'
when serializing sklearn.ensemble._forest.RandomForestRegressor state
when serializing sklearn.ensemble._forest.RandomForestRegressor object
when serializing tuple item 1
when serializing list item 1
when serializing dict item 'steps'
when serializing sklearn.pipeline.Pipeline state
```

when serializing sklearn.pipeline.Pipeline object

now trying ridge and lasso regression

Trying to find the best alpha for ridge regression

```
[7]: import pandas as pd
import numpy as np
import joblib

from sklearn.preprocessing import StandardScaler, OneHotEncoder,
    FunctionTransformer
from sklearn.model_selection import train_test_split
from sklearn.linear_model import Ridge
from sklearn.metrics import r2_score, mean_absolute_error
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer

df = pd.read_csv("car.csv")

X = df.drop("selling_price", axis=1)
y = df["selling_price"]

y_log = np.log1p(y)

X_train, X_test, y_train_log, y_test_log = train_test_split(
    X, y_log, test_size=0.2, random_state=42
)

preprocessor = ColumnTransformer([
    ("onehot", OneHotEncoder(sparse_output=False, handle_unknown="ignore"),
    ["car_name", "brand", "model", "fuel_type", "seller_type"]),

    ("log_km", FunctionTransformer(np.log1p),
    ["km_driven"]),

    ("scaler", StandardScaler(),
    ["vehicle_age", "engine", "max_power", "mileage"]),
])

alphas = [0.0001, 0.001, 0.01, 0.1, 1, 10, 100]

best_alpha = None
best_test_r2 = -999
best_pipeline = None

for alpha in alphas:
```

```

pipeline = Pipeline([
    ("preprocessor", preprocessor),
    ("model", Ridge(alpha=alpha))
])

pipeline.fit(X_train, y_train_log)

train_pred_log = pipeline.predict(X_train)
test_pred_log = pipeline.predict(X_test)

train_r2 = r2_score(np.expm1(y_train_log), np.expm1(train_pred_log))
test_r2 = r2_score(np.expm1(y_test_log), np.expm1(test_pred_log))

test_mae = mean_absolute_error(
    np.expm1(y_test_log),
    np.expm1(test_pred_log)
)

print(
    f"Alpha = {alpha:<7} "
    f"Train R2 = {train_r2:.4f}  "
    f"Test R2 = {test_r2:.4f}  "
    f"MAE = {test_mae:,.0f}"
)

if test_r2 > best_test_r2:
    best_test_r2 = test_r2
    best_alpha = alpha
    best_pipeline = pipeline

print("\nBest Alpha:", best_alpha)
print("Best Test R2:", round(best_test_r2, 4))

joblib.dump(best_pipeline, "ridgemodel.pkl")

```

Alpha = 0.0001	Train R ² = 0.9306	Test R ² = 0.9061	MAE = 118,263
Alpha = 0.001	Train R ² = 0.9306	Test R ² = 0.9061	MAE = 118,264
Alpha = 0.01	Train R ² = 0.9306	Test R ² = 0.9061	MAE = 118,266
Alpha = 0.1	Train R ² = 0.9301	Test R ² = 0.9062	MAE = 118,288
Alpha = 1	Train R ² = 0.9133	Test R ² = 0.9050	MAE = 118,801
Alpha = 10	Train R ² = 0.8910	Test R ² = 0.8820	MAE = 126,705
Alpha = 100	Train R ² = 0.6551	Test R ² = 0.7980	MAE = 147,109

Best Alpha: 0.1
Best Test R²: 0.9062

[7]: ['ridgemodel.pkl']

there is only difference of 0.1 in in linear regression model and ridge regssion model

NOW TRAINING AND CHCKING FOFR BEST LAMBDA IN LASSO REGRESSSION

```
[8]: import pandas as pd
import numpy as np
import joblib

from sklearn.preprocessing import StandardScaler, OneHotEncoder, FunctionTransformer
from sklearn.model_selection import train_test_split
from sklearn.linear_model import Lasso
from sklearn.metrics import r2_score, mean_absolute_error
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer

df = pd.read_csv("car.csv")

X = df.drop("selling_price", axis=1)
y = df["selling_price"]

y_log = np.log1p(y)

X_train, X_test, y_train_log, y_test_log = train_test_split(
    X, y_log, test_size=0.2, random_state=42
)

preprocessor = ColumnTransformer([
    ("onehot", OneHotEncoder(sparse_output=False, handle_unknown="ignore"),
     ["car_name", "brand", "model", "fuel_type", "seller_type"]),

    ("log_km", FunctionTransformer(np.log1p),
     ["km_driven"]),

    ("scaler", StandardScaler(),
     ["vehicle_age", "engine", "max_power", "mileage"]),
])

alphas = [0.0001, 0.001, 0.01, 0.1, 1, 10, 100]

best_alpha = None
best_test_r2 = -999
best_pipeline = None

for alpha in alphas:
    pipeline = Pipeline([
        ("preprocessor", preprocessor),
        ("model", Lasso(alpha=alpha, max_iter=10000))
```

```

])

pipeline.fit(X_train, y_train_log)

train_pred_log = pipeline.predict(X_train)
test_pred_log = pipeline.predict(X_test)

train_r2 = r2_score(np.expm1(y_train_log), np.expm1(train_pred_log))
test_r2 = r2_score(np.expm1(y_test_log), np.expm1(test_pred_log))

test_mae = mean_absolute_error(
    np.expm1(y_test_log),
    np.expm1(test_pred_log)
)

print(
    f"Alpha = {alpha:<7} "
    f"Train R² = {train_r2:.4f} "
    f"Test R² = {test_r2:.4f} "
    f"MAE = {test_mae:,.0f}"
)

if test_r2 > best_test_r2:
    best_test_r2 = test_r2
    best_alpha = alpha
    best_pipeline = pipeline

print("\nBest Alpha:", best_alpha)
print("Best Test R²:", round(best_test_r2, 4))

joblib.dump(best_pipeline, "lasssomodel.pkl")

```

Alpha = 0.0001	Train R² = 0.8625	Test R² = 0.8968	MAE = 123,782
Alpha = 0.001	Train R² = 0.5793	Test R² = 0.7487	MAE = 153,457
Alpha = 0.01	Train R² = -2.7602	Test R² = 0.4872	MAE = 189,094
Alpha = 0.1	Train R² = -0.2322	Test R² = 0.6209	MAE = 222,489
Alpha = 1	Train R² = -0.0431	Test R² = -0.0538	MAE = 418,849
Alpha = 10	Train R² = -0.0431	Test R² = -0.0538	MAE = 418,849
Alpha = 100	Train R² = -0.0431	Test R² = -0.0538	MAE = 418,849

```

Best Alpha: 0.0001
Best Test R²: 0.8968

```

```
[8]: ['lasssomodel.pkl']
```

Linear Regression performed better than Lasso on this dataset. The best Lasso alpha was 0.0001, which is very close to zero, indicating that the dataset needs little to no regularization. This suggests that the existing features are already informative, and Lasso's penalty slightly shrinks

useful coefficients, reducing the model's accuracy. Therefore, Linear Regression is the preferred model for this dataset when compared to ridge and lasso.

Now trying xgboost

```
[4]: import pandas as pd
import numpy as np
import joblib
from sklearn.preprocessing import
    ↳StandardScaler, OneHotEncoder, FunctionTransformer
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_absolute_error
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.ensemble import RandomForestRegressor
from xgboost import XGBRegressor

xgboostmodeldf = pd.read_csv("car.csv")
# print(linearmodeldf.columns.tolist())

X = xgboostmodeldf.drop("selling_price", axis=1)
y = xgboostmodeldf["selling_price"]
# log transform target
y_log = np.log1p(y)
X_train, X_test, y_train_log, y_test_log = train_test_split(
X, y_log, test_size=0.2, random_state=42
)
tf1=ColumnTransformer([
    ↳
    ↳("Onehotencoder", OneHotEncoder(sparse_output=False, handle_unknown="ignore"), ["car_name", "br
        ("Logtransformer", FunctionTransformer(np.log1p), ["km_driven"])),
    ↳
    ↳("StandardScaler", StandardScaler(), ["vehicle_age", "engine", "max_power", "mileage"])),
])
pipeline=Pipeline(
    [
        ("preprocessor", tf1),
        (
            "model", XGBRegressor(max_depth = 4)
        )
    ]
)

pipeline.fit(X_train, y_train_log)
```



```

# predict log price

# testing model with custom input

# custom_input = pd.DataFrame({
#     "car_name": ["Honda City"],
#     "brand": ["Honda"],
#     "model": ["City"],
#     "vehicle_age": [5],
#     "km_driven": [45000],
#     "seller_type": ["Dealer"],
#     "fuel_type": ["Petrol"],
#     "engine": [1498],
#     "mileage": [17.8],
#     "max_power": [117.3]
# })

custom_input = pd.DataFrame({
    "car_name": ["Mercedes-Benz C-Class"],
    "brand": ["Mercedes-Benz"],
    "model": ["C-Class"],
    "vehicle_age": [4],
    "km_driven": [38000],
    "seller_type": ["Dealer"],
    "fuel_type": ["Diesel"],
    "engine": [1950],
    "mileage": [19.0],
    "max_power": [191.0]
})

# calculating train r square
train_pred=pipeline.predict(X_train)

pred_log = pipeline.predict(custom_input)
pred_price = np.expm1(pred_log)

print(f"Predicted Price: {pred_price[0]:,.0f}")

pred_log = pipeline.predict(X_test)
# convert back to actual price
pred_price = np.expm1(pred_log)
y_test_price = np.expm1(y_test_log)
print("R2 test Score:", round(r2_score(y_test_price, pred_price), 4))
print("r2 train score", r2_score(np.expm1(train_pred), np.expm1(y_train_log)))
print("MAE:", round(mean_absolute_error(y_test_price, pred_price), 2))

joblib.dump(pipeline, "xgboostmodel.pkl")

```

Predicted Price: 3,550,748
R2 test Score: 0.829
r2 train score 0.9531995541122703
MAE: 107382.03

[4]: ['xgboostmodel.pkl']

xgboost model is overfitting even after reducing max depth now trying to tune it

```
[4]: import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler, OneHotEncoder,
    FunctionTransformer
from sklearn.model_selection import train_test_split, RandomizedSearchCV, KFold
from sklearn.metrics import r2_score, mean_absolute_error
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from xgboost import XGBRegressor

# ----- Load data -----
df = pd.read_csv("car.csv")
X = df.drop("selling_price", axis=1)
y = df["selling_price"]
y_log = np.log1p(y)

X_train, X_test, y_train_log, y_test_log = train_test_split(
    X, y_log, test_size=0.2, random_state=42
)

# ----- Preprocessing -----
tf1 = ColumnTransformer([
    ("Onehotencoder", OneHotEncoder(sparse_output=False,
    handle_unknown="ignore"),
    ["car_name", "brand", "model", "fuel_type",
    "seller_type", "transmission_type"]),
    ("Logtransformer", FunctionTransformer(np.log1p), ["km_driven"]),
    ("StandardScaler", StandardScaler(), ["vehicle_age", "engine", "max_power",
    "mileage"]),
])

# ----- Pipeline (no early stopping here - search handles n_estimators
    directly) -----
pipeline = Pipeline([
    ("preprocessor", tf1),
    ("model", XGBRegressor(random_state=42, eval_metric="rmse")),
])

# ----- Parameter search space -----
```

```

param_dist = {
    "model__n_estimators": [200, 400, 600, 800, 1000],
    "model__max_depth": [3, 4, 5, 6, 7],
    "model__learning_rate": [0.01, 0.03, 0.05, 0.07, 0.1],
    "model__subsample": [0.6, 0.7, 0.8, 0.9, 1.0],
    "model__colsample_bytree": [0.4, 0.5, 0.6, 0.7, 0.8],
    "model__colsample_bylevel": [0.4, 0.5, 0.6, 0.7, 0.8],
    "model__min_child_weight": [1, 3, 5, 7, 10],
    "model__reg_alpha": [0, 0.01, 0.1, 0.5, 1, 2],
    "model__reg_lambda": [0.5, 1, 5, 10, 20],
    "model__gamma": [0, 0.1, 0.3, 0.5, 1],
}

search = RandomizedSearchCV(
    pipeline,
    param_distributions=param_dist,
    n_iter=50, # number of random combos to try - raise for a
    ↪more thorough search
    scoring="r2",
    cv=KFold(n_splits=5, shuffle=True, random_state=42),
    random_state=42,
    n_jobs=-1,
    verbose=2,
)

search.fit(X_train, y_train_log)

print("\nBest params:", search.best_params_)
print("Best CV R2:", search.best_score_)

# ----- Evaluate best model on held-out test set -----
best_pipeline = search.best_estimator_

train_pred = np.expm1(best_pipeline.predict(X_train))
test_pred = np.expm1(best_pipeline.predict(X_test))
y_train_price = np.expm1(y_train_log)
y_test_price = np.expm1(y_test_log)

print("\n--- Final tuned XGBoost ---")
print("R2 train:", round(r2_score(y_train_price, train_pred), 4))
print("R2 test:", round(r2_score(y_test_price, test_pred), 4))
print("MAE:", round(mean_absolute_error(y_test_price, test_pred), 2))

import joblib
joblib.dump(best_pipeline, "xgboostmodel_tuned.pkl")
print("\nSaved best pipeline to xgboostmodel_tuned.pkl")

```

Fitting 5 folds for each of 50 candidates, totalling 250 fits

[CV] END model__colsample_bylevel=0.6, model__colsample_bytree=0.7,
model__gamma=0.5, model__learning_rate=0.05, model__max_depth=5,
model__min_child_weight=10, model__n_estimators=600, model__reg_alpha=0.5,
model__reg_lambda=5, model__subsample=0.9; total time= 17.2s

[CV] END model__colsample_bylevel=0.6, model__colsample_bytree=0.7,
model__gamma=0.5, model__learning_rate=0.05, model__max_depth=5,
model__min_child_weight=10, model__n_estimators=600, model__reg_alpha=0.5,
model__reg_lambda=5, model__subsample=0.9; total time= 19.3s

[CV] END model__colsample_bylevel=0.6, model__colsample_bytree=0.7,
model__gamma=0.5, model__learning_rate=0.05, model__max_depth=5,
model__min_child_weight=10, model__n_estimators=600, model__reg_alpha=0.5,
model__reg_lambda=5, model__subsample=0.9; total time= 23.1s

[CV] END model__colsample_bylevel=0.6, model__colsample_bytree=0.7,
model__gamma=0.5, model__learning_rate=0.05, model__max_depth=5,
model__min_child_weight=10, model__n_estimators=600, model__reg_alpha=0.5,
model__reg_lambda=5, model__subsample=0.9; total time= 22.5s

[CV] END model__colsample_bylevel=0.6, model__colsample_bytree=0.7,
model__gamma=1, model__learning_rate=0.1, model__max_depth=4,
model__min_child_weight=10, model__n_estimators=200, model__reg_alpha=2,
model__reg_lambda=1, model__subsample=1.0; total time= 6.8s

[CV] END model__colsample_bylevel=0.6, model__colsample_bytree=0.7,
model__gamma=1, model__learning_rate=0.1, model__max_depth=4,
model__min_child_weight=10, model__n_estimators=200, model__reg_alpha=2,
model__reg_lambda=1, model__subsample=1.0; total time= 7.9s

[CV] END model__colsample_bylevel=0.6, model__colsample_bytree=0.7,
model__gamma=1, model__learning_rate=0.1, model__max_depth=4,
model__min_child_weight=10, model__n_estimators=200, model__reg_alpha=2,
model__reg_lambda=1, model__subsample=1.0; total time= 8.9s

[CV] END model__colsample_bylevel=0.6, model__colsample_bytree=0.7,
model__gamma=1, model__learning_rate=0.1, model__max_depth=4,
model__min_child_weight=10, model__n_estimators=200, model__reg_alpha=2,
model__reg_lambda=1, model__subsample=1.0; total time= 6.7s

[CV] END model__colsample_bylevel=0.6, model__colsample_bytree=0.7,
model__gamma=0.5, model__learning_rate=0.05, model__max_depth=5,
model__min_child_weight=10, model__n_estimators=600, model__reg_alpha=0.5,
model__reg_lambda=5, model__subsample=0.9; total time= 17.0s

[CV] END model__colsample_bylevel=0.6, model__colsample_bytree=0.7,
model__gamma=1, model__learning_rate=0.1, model__max_depth=4,
model__min_child_weight=10, model__n_estimators=200, model__reg_alpha=2,
model__reg_lambda=1, model__subsample=1.0; total time= 6.6s

[CV] END model__colsample_bylevel=0.8, model__colsample_bytree=0.5,
model__gamma=0.3, model__learning_rate=0.05, model__max_depth=6,
model__min_child_weight=3, model__n_estimators=1000, model__reg_alpha=0,
model__reg_lambda=0.5, model__subsample=0.7; total time= 19.4s

[CV] END model__colsample_bylevel=0.8, model__colsample_bytree=0.5,
model__gamma=0.3, model__learning_rate=0.05, model__max_depth=6,
model__min_child_weight=3, model__n_estimators=1000, model__reg_alpha=0,

```

model__reg_lambda=0.5, model__subsample=0.7; total time= 23.7s
[CV] END model__colsample_bylevel=0.8, model__colsample_bytree=0.5,
model__gamma=0.3, model__learning_rate=0.05, model__max_depth=6,
model__min_child_weight=3, model__n_estimators=1000, model__reg_alpha=0,
model__reg_lambda=0.5, model__subsample=0.7; total time= 23.0s
[CV] END model__colsample_bylevel=0.8, model__colsample_bytree=0.5,
model__gamma=0.3, model__learning_rate=0.05, model__max_depth=6,
model__min_child_weight=3, model__n_estimators=1000, model__reg_alpha=0,
model__reg_lambda=0.5, model__subsample=0.7; total time= 24.8s
[CV] END model__colsample_bylevel=0.5, model__colsample_bytree=0.8,
model__gamma=0, model__learning_rate=0.1, model__max_depth=5,
model__min_child_weight=10, model__n_estimators=600, model__reg_alpha=0,
model__reg_lambda=20, model__subsample=0.8; total time= 17.2s
[CV] END model__colsample_bylevel=0.8, model__colsample_bytree=0.5,
model__gamma=0.3, model__learning_rate=0.05, model__max_depth=6,
model__min_child_weight=3, model__n_estimators=1000, model__reg_alpha=0,
model__reg_lambda=0.5, model__subsample=0.7; total time= 21.8s
[CV] END model__colsample_bylevel=0.5, model__colsample_bytree=0.8,
model__gamma=0, model__learning_rate=0.1, model__max_depth=5,
model__min_child_weight=10, model__n_estimators=600, model__reg_alpha=0,
model__reg_lambda=20, model__subsample=0.8; total time= 17.1s
[CV] END model__colsample_bylevel=0.5, model__colsample_bytree=0.8,
model__gamma=0, model__learning_rate=0.1, model__max_depth=5,
model__min_child_weight=10, model__n_estimators=600, model__reg_alpha=0,
model__reg_lambda=20, model__subsample=0.8; total time= 18.5s
[CV] END model__colsample_bylevel=0.4, model__colsample_bytree=0.8,
model__gamma=0.5, model__learning_rate=0.1, model__max_depth=3,
model__min_child_weight=10, model__n_estimators=400, model__reg_alpha=0.5,
model__reg_lambda=5, model__subsample=1.0; total time= 7.4s
[CV] END model__colsample_bylevel=0.5, model__colsample_bytree=0.8,
model__gamma=0, model__learning_rate=0.1, model__max_depth=5,
model__min_child_weight=10, model__n_estimators=600, model__reg_alpha=0,
model__reg_lambda=20, model__subsample=0.8; total time= 14.9s
[CV] END model__colsample_bylevel=0.4, model__colsample_bytree=0.8,
model__gamma=0.5, model__learning_rate=0.1, model__max_depth=3,
model__min_child_weight=10, model__n_estimators=400, model__reg_alpha=0.5,
model__reg_lambda=5, model__subsample=1.0; total time= 8.7s
[CV] END model__colsample_bylevel=0.4, model__colsample_bytree=0.8,
model__gamma=0.5, model__learning_rate=0.1, model__max_depth=3,
model__min_child_weight=10, model__n_estimators=400, model__reg_alpha=0.5,
model__reg_lambda=5, model__subsample=1.0; total time= 8.0s
[CV] END model__colsample_bylevel=0.5, model__colsample_bytree=0.8,
model__gamma=0, model__learning_rate=0.1, model__max_depth=5,
model__min_child_weight=10, model__n_estimators=600, model__reg_alpha=0,
model__reg_lambda=20, model__subsample=0.8; total time= 18.8s
[CV] END model__colsample_bylevel=0.4, model__colsample_bytree=0.8,
model__gamma=0.5, model__learning_rate=0.1, model__max_depth=3,
model__min_child_weight=10, model__n_estimators=400, model__reg_alpha=0.5,

```

```

model__reg_lambda=5, model__subsample=1.0; total time= 7.6s
[CV] END model__colsample_bylevel=0.8, model__colsample_bytree=0.5,
model__gamma=0.1, model__learning_rate=0.03, model__max_depth=3,
model__min_child_weight=7, model__n_estimators=200, model__reg_alpha=1,
model__reg_lambda=5, model__subsample=1.0; total time= 8.3s
[CV] END model__colsample_bylevel=0.4, model__colsample_bytree=0.8,
model__gamma=0.5, model__learning_rate=0.1, model__max_depth=3,
model__min_child_weight=10, model__n_estimators=400, model__reg_alpha=0.5,
model__reg_lambda=5, model__subsample=1.0; total time= 9.5s
[CV] END model__colsample_bylevel=0.8, model__colsample_bytree=0.5,
model__gamma=0.1, model__learning_rate=0.03, model__max_depth=3,
model__min_child_weight=7, model__n_estimators=200, model__reg_alpha=1,
model__reg_lambda=5, model__subsample=1.0; total time= 6.3s
[CV] END model__colsample_bylevel=0.8, model__colsample_bytree=0.5,
model__gamma=0.1, model__learning_rate=0.03, model__max_depth=3,
model__min_child_weight=7, model__n_estimators=200, model__reg_alpha=1,
model__reg_lambda=5, model__subsample=1.0; total time= 9.3s
[CV] END model__colsample_bylevel=0.8, model__colsample_bytree=0.5,
model__gamma=0.1, model__learning_rate=0.03, model__max_depth=3,
model__min_child_weight=7, model__n_estimators=200, model__reg_alpha=1,
model__reg_lambda=5, model__subsample=1.0; total time= 7.6s
[CV] END model__colsample_bylevel=0.8, model__colsample_bytree=0.5,
model__gamma=0.1, model__learning_rate=0.03, model__max_depth=3,
model__min_child_weight=7, model__n_estimators=200, model__reg_alpha=1,
model__reg_lambda=5, model__subsample=1.0; total time= 8.1s
[CV] END model__colsample_bylevel=0.8, model__colsample_bytree=0.4,
model__gamma=0.1, model__learning_rate=0.05, model__max_depth=7,
model__min_child_weight=10, model__n_estimators=600, model__reg_alpha=2,
model__reg_lambda=1, model__subsample=0.8; total time= 16.2s
[CV] END model__colsample_bylevel=0.8, model__colsample_bytree=0.4,
model__gamma=0.1, model__learning_rate=0.05, model__max_depth=7,
model__min_child_weight=10, model__n_estimators=600, model__reg_alpha=2,
model__reg_lambda=1, model__subsample=0.8; total time= 21.1s
[CV] END model__colsample_bylevel=0.8, model__colsample_bytree=0.4,
model__gamma=0.1, model__learning_rate=0.05, model__max_depth=7,
model__min_child_weight=10, model__n_estimators=600, model__reg_alpha=2,
model__reg_lambda=1, model__subsample=0.8; total time= 16.7s
[CV] END model__colsample_bylevel=0.8, model__colsample_bytree=0.4,
model__gamma=0.1, model__learning_rate=0.05, model__max_depth=7,
model__min_child_weight=10, model__n_estimators=600, model__reg_alpha=2,
model__reg_lambda=1, model__subsample=0.8; total time= 20.0s
[CV] END model__colsample_bylevel=0.8, model__colsample_bytree=0.4,
model__gamma=0.1, model__learning_rate=0.05, model__max_depth=7,
model__min_child_weight=10, model__n_estimators=600, model__reg_alpha=2,
model__reg_lambda=1, model__subsample=0.8; total time= 19.1s
[CV] END model__colsample_bylevel=0.7, model__colsample_bytree=0.4,
model__gamma=0.1, model__learning_rate=0.1, model__max_depth=4,
model__min_child_weight=1, model__n_estimators=800, model__reg_alpha=0,

```

model__reg_lambda=5, model__subsample=0.8; total time= 16.7s
 [CV] END model__colsample_bylevel=0.7, model__colsample_bytree=0.4,
 model__gamma=0.1, model__learning_rate=0.1, model__max_depth=4,
 model__min_child_weight=1, model__n_estimators=800, model__reg_alpha=0,
 model__reg_lambda=5, model__subsample=0.8; total time= 17.2s
 [CV] END model__colsample_bylevel=0.7, model__colsample_bytree=0.4,
 model__gamma=0.1, model__learning_rate=0.1, model__max_depth=4,
 model__min_child_weight=1, model__n_estimators=800, model__reg_alpha=0,
 model__reg_lambda=5, model__subsample=0.8; total time= 14.0s
 [CV] END model__colsample_bylevel=0.8, model__colsample_bytree=0.4,
 model__gamma=0.3, model__learning_rate=0.07, model__max_depth=5,
 model__min_child_weight=7, model__n_estimators=200, model__reg_alpha=0,
 model__reg_lambda=10, model__subsample=1.0; total time= 5.7s
 [CV] END model__colsample_bylevel=0.8, model__colsample_bytree=0.4,
 model__gamma=0.3, model__learning_rate=0.07, model__max_depth=5,
 model__min_child_weight=7, model__n_estimators=200, model__reg_alpha=0,
 model__reg_lambda=10, model__subsample=1.0; total time= 6.3s
 [CV] END model__colsample_bylevel=0.8, model__colsample_bytree=0.4,
 model__gamma=0.3, model__learning_rate=0.07, model__max_depth=5,
 model__min_child_weight=7, model__n_estimators=200, model__reg_alpha=0,
 model__reg_lambda=10, model__subsample=1.0; total time= 4.9s
 [CV] END model__colsample_bylevel=0.8, model__colsample_bytree=0.4,
 model__gamma=0.3, model__learning_rate=0.07, model__max_depth=5,
 model__min_child_weight=7, model__n_estimators=200, model__reg_alpha=0,
 model__reg_lambda=10, model__subsample=1.0; total time= 7.7s
 [CV] END model__colsample_bylevel=0.8, model__colsample_bytree=0.4,
 model__gamma=0.3, model__learning_rate=0.07, model__max_depth=5,
 model__min_child_weight=7, model__n_estimators=200, model__reg_alpha=0,
 model__reg_lambda=10, model__subsample=1.0; total time= 7.0s
 [CV] END model__colsample_bylevel=0.7, model__colsample_bytree=0.4,
 model__gamma=0.1, model__learning_rate=0.1, model__max_depth=4,
 model__min_child_weight=1, model__n_estimators=800, model__reg_alpha=0,
 model__reg_lambda=5, model__subsample=0.8; total time= 21.6s
 [CV] END model__colsample_bylevel=0.7, model__colsample_bytree=0.4,
 model__gamma=0.1, model__learning_rate=0.1, model__max_depth=4,
 model__min_child_weight=1, model__n_estimators=800, model__reg_alpha=0,
 model__reg_lambda=5, model__subsample=0.8; total time= 20.4s
 [CV] END model__colsample_bylevel=0.5, model__colsample_bytree=0.8,
 model__gamma=0.3, model__learning_rate=0.07, model__max_depth=5,
 model__min_child_weight=7, model__n_estimators=400, model__reg_alpha=0.1,
 model__reg_lambda=20, model__subsample=0.7; total time= 11.7s
 [CV] END model__colsample_bylevel=0.5, model__colsample_bytree=0.8,
 model__gamma=0.3, model__learning_rate=0.07, model__max_depth=5,
 model__min_child_weight=7, model__n_estimators=400, model__reg_alpha=0.1,
 model__reg_lambda=20, model__subsample=0.7; total time= 11.4s
 [CV] END model__colsample_bylevel=0.5, model__colsample_bytree=0.8,
 model__gamma=0.3, model__learning_rate=0.07, model__max_depth=5,
 model__min_child_weight=7, model__n_estimators=400, model__reg_alpha=0.1,

```

model__reg_lambda=20, model__subsample=0.7; total time= 11.9s
[CV] END model__colsample_bylevel=0.5, model__colsample_bytree=0.8,
model__gamma=0.3, model__learning_rate=0.07, model__max_depth=5,
model__min_child_weight=7, model__n_estimators=400, model__reg_alpha=0.1,
model__reg_lambda=20, model__subsample=0.7; total time= 11.6s
[CV] END model__colsample_bylevel=0.5, model__colsample_bytree=0.8,
model__gamma=0.3, model__learning_rate=0.07, model__max_depth=5,
model__min_child_weight=7, model__n_estimators=400, model__reg_alpha=0.1,
model__reg_lambda=20, model__subsample=0.7; total time= 11.1s
[CV] END model__colsample_bylevel=0.8, model__colsample_bytree=0.6,
model__gamma=0.3, model__learning_rate=0.03, model__max_depth=5,
model__min_child_weight=1, model__n_estimators=1000, model__reg_alpha=0,
model__reg_lambda=0.5, model__subsample=0.6; total time= 19.6s
[CV] END model__colsample_bylevel=0.8, model__colsample_bytree=0.6,
model__gamma=0.3, model__learning_rate=0.03, model__max_depth=5,
model__min_child_weight=1, model__n_estimators=1000, model__reg_alpha=0,
model__reg_lambda=0.5, model__subsample=0.6; total time= 21.5s
[CV] END model__colsample_bylevel=0.8, model__colsample_bytree=0.6,
model__gamma=0.3, model__learning_rate=0.03, model__max_depth=5,
model__min_child_weight=1, model__n_estimators=1000, model__reg_alpha=0,
model__reg_lambda=0.5, model__subsample=0.6; total time= 23.5s
[CV] END model__colsample_bylevel=0.8, model__colsample_bytree=0.6,
model__gamma=0.3, model__learning_rate=0.03, model__max_depth=5,
model__min_child_weight=1, model__n_estimators=1000, model__reg_alpha=0,
model__reg_lambda=0.5, model__subsample=0.6; total time= 21.1s
[CV] END model__colsample_bylevel=0.5, model__colsample_bytree=0.8,
model__gamma=0.5, model__learning_rate=0.03, model__max_depth=4,
model__min_child_weight=3, model__n_estimators=600, model__reg_alpha=1,
model__reg_lambda=10, model__subsample=1.0; total time= 15.2s
[CV] END model__colsample_bylevel=0.5, model__colsample_bytree=0.8,
model__gamma=0.5, model__learning_rate=0.03, model__max_depth=4,
model__min_child_weight=3, model__n_estimators=600, model__reg_alpha=1,
model__reg_lambda=10, model__subsample=1.0; total time= 16.2s
[CV] END model__colsample_bylevel=0.5, model__colsample_bytree=0.8,
model__gamma=0.5, model__learning_rate=0.03, model__max_depth=4,
model__min_child_weight=3, model__n_estimators=600, model__reg_alpha=1,
model__reg_lambda=10, model__subsample=1.0; total time= 15.8s
[CV] END model__colsample_bylevel=0.8, model__colsample_bytree=0.6,
model__gamma=0.3, model__learning_rate=0.03, model__max_depth=5,
model__min_child_weight=1, model__n_estimators=1000, model__reg_alpha=0,
model__reg_lambda=0.5, model__subsample=0.6; total time= 25.9s
[CV] END model__colsample_bylevel=0.7, model__colsample_bytree=0.4,
model__gamma=0.5, model__learning_rate=0.1, model__max_depth=7,
model__min_child_weight=7, model__n_estimators=200, model__reg_alpha=1,
model__reg_lambda=0.5, model__subsample=0.8; total time= 4.3s
[CV] END model__colsample_bylevel=0.5, model__colsample_bytree=0.8,
model__gamma=0.5, model__learning_rate=0.03, model__max_depth=4,
model__min_child_weight=3, model__n_estimators=600, model__reg_alpha=1,

```


model__reg_lambda=10, model__subsample=1.0; total time= 12.8s
 [CV] END model__colsample_bylevel=0.5, model__colsample_bytree=0.8,
 model__gamma=0.5, model__learning_rate=0.03, model__max_depth=4,
 model__min_child_weight=3, model__n_estimators=600, model__reg_alpha=1,
 model__reg_lambda=10, model__subsample=1.0; total time= 15.6s
 [CV] END model__colsample_bylevel=0.7, model__colsample_bytree=0.4,
 model__gamma=0.5, model__learning_rate=0.1, model__max_depth=7,
 model__min_child_weight=7, model__n_estimators=200, model__reg_alpha=1,
 model__reg_lambda=0.5, model__subsample=0.8; total time= 5.1s
 [CV] END model__colsample_bylevel=0.7, model__colsample_bytree=0.4,
 model__gamma=0.5, model__learning_rate=0.1, model__max_depth=7,
 model__min_child_weight=7, model__n_estimators=200, model__reg_alpha=1,
 model__reg_lambda=0.5, model__subsample=0.8; total time= 6.8s
 [CV] END model__colsample_bylevel=0.7, model__colsample_bytree=0.4,
 model__gamma=0.5, model__learning_rate=0.1, model__max_depth=7,
 model__min_child_weight=7, model__n_estimators=200, model__reg_alpha=1,
 model__reg_lambda=0.5, model__subsample=0.8; total time= 4.3s
 [CV] END model__colsample_bylevel=0.7, model__colsample_bytree=0.4,
 model__gamma=0.5, model__learning_rate=0.1, model__max_depth=7,
 model__min_child_weight=7, model__n_estimators=200, model__reg_alpha=1,
 model__reg_lambda=0.5, model__subsample=0.8; total time= 6.8s
 [CV] END model__colsample_bylevel=0.6, model__colsample_bytree=0.4,
 model__gamma=0.5, model__learning_rate=0.01, model__max_depth=6,
 model__min_child_weight=5, model__n_estimators=200, model__reg_alpha=0,
 model__reg_lambda=1, model__subsample=0.6; total time= 9.3s
 [CV] END model__colsample_bylevel=0.6, model__colsample_bytree=0.4,
 model__gamma=0.5, model__learning_rate=0.01, model__max_depth=6,
 model__min_child_weight=5, model__n_estimators=200, model__reg_alpha=0,
 model__reg_lambda=1, model__subsample=0.6; total time= 9.8s
 [CV] END model__colsample_bylevel=0.6, model__colsample_bytree=0.4,
 model__gamma=0.5, model__learning_rate=0.01, model__max_depth=6,
 model__min_child_weight=5, model__n_estimators=200, model__reg_alpha=0,
 model__reg_lambda=1, model__subsample=0.6; total time= 8.5s
 [CV] END model__colsample_bylevel=0.6, model__colsample_bytree=0.4,
 model__gamma=0.5, model__learning_rate=0.01, model__max_depth=6,
 model__min_child_weight=5, model__n_estimators=200, model__reg_alpha=0,
 model__reg_lambda=1, model__subsample=0.6; total time= 12.1s
 [CV] END model__colsample_bylevel=0.6, model__colsample_bytree=0.4,
 model__gamma=0.5, model__learning_rate=0.01, model__max_depth=6,
 model__min_child_weight=5, model__n_estimators=200, model__reg_alpha=0,
 model__reg_lambda=1, model__subsample=0.6; total time= 9.9s
 [CV] END model__colsample_bylevel=0.4, model__colsample_bytree=0.7,
 model__gamma=0.5, model__learning_rate=0.1, model__max_depth=4,
 model__min_child_weight=1, model__n_estimators=1000, model__reg_alpha=0.01,
 model__reg_lambda=10, model__subsample=0.7; total time= 15.2s
 [CV] END model__colsample_bylevel=0.4, model__colsample_bytree=0.7,
 model__gamma=0.5, model__learning_rate=0.1, model__max_depth=4,
 model__min_child_weight=1, model__n_estimators=1000, model__reg_alpha=0.01,

model__reg_lambda=10, model__subsample=0.7; total time= 19.0s
 [CV] END model__colsample_bylevel=0.4, model__colsample_bytree=0.7,
 model__gamma=0.5, model__learning_rate=0.1, model__max_depth=4,
 model__min_child_weight=1, model__n_estimators=1000, model__reg_alpha=0.01,
 model__reg_lambda=10, model__subsample=0.7; total time= 18.3s
 [CV] END model__colsample_bylevel=0.4, model__colsample_bytree=0.7,
 model__gamma=0.5, model__learning_rate=0.1, model__max_depth=4,
 model__min_child_weight=1, model__n_estimators=1000, model__reg_alpha=0.01,
 model__reg_lambda=10, model__subsample=0.7; total time= 20.1s
 [CV] END model__colsample_bylevel=0.4, model__colsample_bytree=0.7,
 model__gamma=0.5, model__learning_rate=0.1, model__max_depth=4,
 model__min_child_weight=1, model__n_estimators=1000, model__reg_alpha=0.01,
 model__reg_lambda=10, model__subsample=0.7; total time= 17.9s
 [CV] END model__colsample_bylevel=0.5, model__colsample_bytree=0.8,
 model__gamma=0.5, model__learning_rate=0.03, model__max_depth=3,
 model__min_child_weight=7, model__n_estimators=600, model__reg_alpha=0.1,
 model__reg_lambda=20, model__subsample=0.9; total time= 12.2s
 [CV] END model__colsample_bylevel=0.5, model__colsample_bytree=0.8,
 model__gamma=0.5, model__learning_rate=0.03, model__max_depth=3,
 model__min_child_weight=7, model__n_estimators=600, model__reg_alpha=0.1,
 model__reg_lambda=20, model__subsample=0.9; total time= 10.6s
 [CV] END model__colsample_bylevel=0.5, model__colsample_bytree=0.8,
 model__gamma=0.5, model__learning_rate=0.03, model__max_depth=3,
 model__min_child_weight=7, model__n_estimators=600, model__reg_alpha=0.1,
 model__reg_lambda=20, model__subsample=0.9; total time= 14.0s
 [CV] END model__colsample_bylevel=0.5, model__colsample_bytree=0.8,
 model__gamma=0.5, model__learning_rate=0.03, model__max_depth=3,
 model__min_child_weight=7, model__n_estimators=600, model__reg_alpha=0.1,
 model__reg_lambda=20, model__subsample=0.9; total time= 13.1s
 [CV] END model__colsample_bylevel=0.5, model__colsample_bytree=0.8,
 model__gamma=0.5, model__learning_rate=0.03, model__max_depth=3,
 model__min_child_weight=7, model__n_estimators=600, model__reg_alpha=0.1,
 model__reg_lambda=20, model__subsample=0.9; total time= 12.8s
 [CV] END model__colsample_bylevel=0.6, model__colsample_bytree=0.6,
 model__gamma=1, model__learning_rate=0.03, model__max_depth=3,
 model__min_child_weight=3, model__n_estimators=600, model__reg_alpha=0.5,
 model__reg_lambda=0.5, model__subsample=0.8; total time= 12.5s
 [CV] END model__colsample_bylevel=0.6, model__colsample_bytree=0.6,
 model__gamma=1, model__learning_rate=0.03, model__max_depth=3,
 model__min_child_weight=3, model__n_estimators=600, model__reg_alpha=0.5,
 model__reg_lambda=0.5, model__subsample=0.8; total time= 15.9s
 [CV] END model__colsample_bylevel=0.6, model__colsample_bytree=0.6,
 model__gamma=1, model__learning_rate=0.03, model__max_depth=3,
 model__min_child_weight=3, model__n_estimators=600, model__reg_alpha=0.5,
 model__reg_lambda=0.5, model__subsample=0.8; total time= 15.9s
 [CV] END model__colsample_bylevel=0.6, model__colsample_bytree=0.6,
 model__gamma=1, model__learning_rate=0.03, model__max_depth=3,
 model__min_child_weight=3, model__n_estimators=600, model__reg_alpha=0.5,

model__reg_lambda=0.5, model__subsample=0.8; total time= 16.7s
 [CV] END model__colsample_bylevel=0.6, model__colsample_bytree=0.6,
 model__gamma=1, model__learning_rate=0.03, model__max_depth=3,
 model__min_child_weight=3, model__n_estimators=600, model__reg_alpha=0.5,
 model__reg_lambda=0.5, model__subsample=0.8; total time= 17.7s
 [CV] END model__colsample_bylevel=0.5, model__colsample_bytree=0.6,
 model__gamma=0, model__learning_rate=0.07, model__max_depth=4,
 model__min_child_weight=10, model__n_estimators=800, model__reg_alpha=0.1,
 model__reg_lambda=10, model__subsample=1.0; total time= 17.7s
 [CV] END model__colsample_bylevel=0.5, model__colsample_bytree=0.6,
 model__gamma=0, model__learning_rate=0.07, model__max_depth=4,
 model__min_child_weight=10, model__n_estimators=800, model__reg_alpha=0.1,
 model__reg_lambda=10, model__subsample=1.0; total time= 17.6s
 [CV] END model__colsample_bylevel=0.5, model__colsample_bytree=0.6,
 model__gamma=0, model__learning_rate=0.07, model__max_depth=4,
 model__min_child_weight=10, model__n_estimators=800, model__reg_alpha=0.1,
 model__reg_lambda=10, model__subsample=1.0; total time= 20.4s
 [CV] END model__colsample_bylevel=0.5, model__colsample_bytree=0.6,
 model__gamma=0, model__learning_rate=0.07, model__max_depth=4,
 model__min_child_weight=10, model__n_estimators=800, model__reg_alpha=0.1,
 model__reg_lambda=10, model__subsample=1.0; total time= 22.8s
 [CV] END model__colsample_bylevel=0.7, model__colsample_bytree=0.8,
 model__gamma=0.3, model__learning_rate=0.1, model__max_depth=6,
 model__min_child_weight=1, model__n_estimators=600, model__reg_alpha=0.1,
 model__reg_lambda=0.5, model__subsample=0.7; total time= 13.3s
 [CV] END model__colsample_bylevel=0.5, model__colsample_bytree=0.6,
 model__gamma=0, model__learning_rate=0.07, model__max_depth=4,
 model__min_child_weight=10, model__n_estimators=800, model__reg_alpha=0.1,
 model__reg_lambda=10, model__subsample=1.0; total time= 18.9s
 [CV] END model__colsample_bylevel=0.7, model__colsample_bytree=0.8,
 model__gamma=0.3, model__learning_rate=0.1, model__max_depth=6,
 model__min_child_weight=1, model__n_estimators=600, model__reg_alpha=0.1,
 model__reg_lambda=0.5, model__subsample=0.7; total time= 12.6s
 [CV] END model__colsample_bylevel=0.7, model__colsample_bytree=0.8,
 model__gamma=0.3, model__learning_rate=0.1, model__max_depth=6,
 model__min_child_weight=1, model__n_estimators=600, model__reg_alpha=0.1,
 model__reg_lambda=0.5, model__subsample=0.7; total time= 11.6s
 [CV] END model__colsample_bylevel=0.7, model__colsample_bytree=0.8,
 model__gamma=0.3, model__learning_rate=0.1, model__max_depth=6,
 model__min_child_weight=1, model__n_estimators=600, model__reg_alpha=0.1,
 model__reg_lambda=0.5, model__subsample=0.7; total time= 12.9s
 [CV] END model__colsample_bylevel=0.7, model__colsample_bytree=0.8,
 model__gamma=0.3, model__learning_rate=0.1, model__max_depth=6,
 model__min_child_weight=1, model__n_estimators=600, model__reg_alpha=0.1,
 model__reg_lambda=0.5, model__subsample=0.7; total time= 13.2s
 [CV] END model__colsample_bylevel=0.4, model__colsample_bytree=0.4,
 model__gamma=0.1, model__learning_rate=0.01, model__max_depth=5,
 model__min_child_weight=5, model__n_estimators=1000, model__reg_alpha=0,

model__reg_lambda=1, model__subsample=0.6; total time= 28.2s
 [CV] END model__colsample_bylevel=0.4, model__colsample_bytree=0.4,
 model__gamma=0.1, model__learning_rate=0.01, model__max_depth=5,
 model__min_child_weight=5, model__n_estimators=1000, model__reg_alpha=0,
 model__reg_lambda=1, model__subsample=0.6; total time= 30.0s
 [CV] END model__colsample_bylevel=0.4, model__colsample_bytree=0.4,
 model__gamma=0.1, model__learning_rate=0.01, model__max_depth=5,
 model__min_child_weight=5, model__n_estimators=1000, model__reg_alpha=0,
 model__reg_lambda=1, model__subsample=0.6; total time= 29.4s
 [CV] END model__colsample_bylevel=0.4, model__colsample_bytree=0.4,
 model__gamma=0.1, model__learning_rate=0.01, model__max_depth=5,
 model__min_child_weight=5, model__n_estimators=1000, model__reg_alpha=0,
 model__reg_lambda=1, model__subsample=0.6; total time= 30.7s
 [CV] END model__colsample_bylevel=0.4, model__colsample_bytree=0.6,
 model__gamma=0.5, model__learning_rate=0.05, model__max_depth=4,
 model__min_child_weight=7, model__n_estimators=800, model__reg_alpha=0.1,
 model__reg_lambda=0.5, model__subsample=0.8; total time= 15.4s
 [CV] END model__colsample_bylevel=0.4, model__colsample_bytree=0.6,
 model__gamma=0.5, model__learning_rate=0.05, model__max_depth=4,
 model__min_child_weight=7, model__n_estimators=800, model__reg_alpha=0.1,
 model__reg_lambda=0.5, model__subsample=0.8; total time= 14.2s
 [CV] END model__colsample_bylevel=0.4, model__colsample_bytree=0.6,
 model__gamma=0.5, model__learning_rate=0.05, model__max_depth=4,
 model__min_child_weight=7, model__n_estimators=800, model__reg_alpha=0.1,
 model__reg_lambda=0.5, model__subsample=0.8; total time= 16.8s
 [CV] END model__colsample_bylevel=0.4, model__colsample_bytree=0.4,
 model__gamma=0.1, model__learning_rate=0.01, model__max_depth=5,
 model__min_child_weight=5, model__n_estimators=1000, model__reg_alpha=0,
 model__reg_lambda=1, model__subsample=0.6; total time= 31.8s
 [CV] END model__colsample_bylevel=0.8, model__colsample_bytree=0.4,
 model__gamma=1, model__learning_rate=0.03, model__max_depth=7,
 model__min_child_weight=3, model__n_estimators=200, model__reg_alpha=2,
 model__reg_lambda=1, model__subsample=0.9; total time= 5.6s
 [CV] END model__colsample_bylevel=0.8, model__colsample_bytree=0.4,
 model__gamma=1, model__learning_rate=0.03, model__max_depth=7,
 model__min_child_weight=3, model__n_estimators=200, model__reg_alpha=2,
 model__reg_lambda=1, model__subsample=0.9; total time= 6.5s
 [CV] END model__colsample_bylevel=0.8, model__colsample_bytree=0.4,
 model__gamma=1, model__learning_rate=0.03, model__max_depth=7,
 model__min_child_weight=3, model__n_estimators=200, model__reg_alpha=2,
 model__reg_lambda=1, model__subsample=0.9; total time= 5.2s
 [CV] END model__colsample_bylevel=0.4, model__colsample_bytree=0.6,
 model__gamma=0.5, model__learning_rate=0.05, model__max_depth=4,
 model__min_child_weight=7, model__n_estimators=800, model__reg_alpha=0.1,
 model__reg_lambda=0.5, model__subsample=0.8; total time= 15.5s
 [CV] END model__colsample_bylevel=0.4, model__colsample_bytree=0.6,
 model__gamma=0.5, model__learning_rate=0.05, model__max_depth=4,
 model__min_child_weight=7, model__n_estimators=800, model__reg_alpha=0.1,

```

model__reg_lambda=0.5, model__subsample=0.8; total time= 16.1s
[CV] END model__colsample_bylevel=0.8, model__colsample_bytree=0.4,
model__gamma=1, model__learning_rate=0.03, model__max_depth=7,
model__min_child_weight=3, model__n_estimators=200, model__reg_alpha=2,
model__reg_lambda=1, model__subsample=0.9; total time= 6.2s
[CV] END model__colsample_bylevel=0.8, model__colsample_bytree=0.4,
model__gamma=1, model__learning_rate=0.03, model__max_depth=7,
model__min_child_weight=3, model__n_estimators=200, model__reg_alpha=2,
model__reg_lambda=1, model__subsample=0.9; total time= 8.1s
[CV] END model__colsample_bylevel=0.6, model__colsample_bytree=0.6,
model__gamma=0, model__learning_rate=0.05, model__max_depth=3,
model__min_child_weight=7, model__n_estimators=200, model__reg_alpha=0.01,
model__reg_lambda=5, model__subsample=1.0; total time= 4.2s
[CV] END model__colsample_bylevel=0.6, model__colsample_bytree=0.6,
model__gamma=0, model__learning_rate=0.05, model__max_depth=3,
model__min_child_weight=7, model__n_estimators=200, model__reg_alpha=0.01,
model__reg_lambda=5, model__subsample=1.0; total time= 4.2s
[CV] END model__colsample_bylevel=0.6, model__colsample_bytree=0.6,
model__gamma=0, model__learning_rate=0.05, model__max_depth=3,
model__min_child_weight=7, model__n_estimators=200, model__reg_alpha=0.01,
model__reg_lambda=5, model__subsample=1.0; total time= 8.4s
[CV] END model__colsample_bylevel=0.6, model__colsample_bytree=0.6,
model__gamma=0, model__learning_rate=0.05, model__max_depth=3,
model__min_child_weight=7, model__n_estimators=200, model__reg_alpha=0.01,
model__reg_lambda=5, model__subsample=1.0; total time= 3.9s
[CV] END model__colsample_bylevel=0.6, model__colsample_bytree=0.6,
model__gamma=0, model__learning_rate=0.05, model__max_depth=3,
model__min_child_weight=7, model__n_estimators=200, model__reg_alpha=0.01,
model__reg_lambda=5, model__subsample=1.0; total time= 6.1s
[CV] END model__colsample_bylevel=0.8, model__colsample_bytree=0.4,
model__gamma=0.5, model__learning_rate=0.05, model__max_depth=3,
model__min_child_weight=5, model__n_estimators=1000, model__reg_alpha=2,
model__reg_lambda=0.5, model__subsample=1.0; total time= 15.7s
[CV] END model__colsample_bylevel=0.8, model__colsample_bytree=0.4,
model__gamma=0.5, model__learning_rate=0.05, model__max_depth=3,
model__min_child_weight=5, model__n_estimators=1000, model__reg_alpha=2,
model__reg_lambda=0.5, model__subsample=1.0; total time= 17.2s
[CV] END model__colsample_bylevel=0.8, model__colsample_bytree=0.4,
model__gamma=0.5, model__learning_rate=0.05, model__max_depth=3,
model__min_child_weight=5, model__n_estimators=1000, model__reg_alpha=2,
model__reg_lambda=0.5, model__subsample=1.0; total time= 18.2s
[CV] END model__colsample_bylevel=0.8, model__colsample_bytree=0.4,
model__gamma=0.5, model__learning_rate=0.05, model__max_depth=3,
model__min_child_weight=5, model__n_estimators=1000, model__reg_alpha=2,
model__reg_lambda=0.5, model__subsample=1.0; total time= 24.2s
[CV] END model__colsample_bylevel=0.7, model__colsample_bytree=0.8,
model__gamma=1, model__learning_rate=0.03, model__max_depth=4,
model__min_child_weight=5, model__n_estimators=600, model__reg_alpha=0.5,

```

model__reg_lambda=20, model__subsample=0.9; total time= 15.1s
 [CV] END model__colsample_bylevel=0.8, model__colsample_bytree=0.4,
 model__gamma=0.5, model__learning_rate=0.05, model__max_depth=3,
 model__min_child_weight=5, model__n_estimators=1000, model__reg_alpha=2,
 model__reg_lambda=0.5, model__subsample=1.0; total time= 17.0s
 [CV] END model__colsample_bylevel=0.7, model__colsample_bytree=0.8,
 model__gamma=1, model__learning_rate=0.03, model__max_depth=4,
 model__min_child_weight=5, model__n_estimators=600, model__reg_alpha=0.5,
 model__reg_lambda=20, model__subsample=0.9; total time= 14.4s
 [CV] END model__colsample_bylevel=0.7, model__colsample_bytree=0.8,
 model__gamma=1, model__learning_rate=0.03, model__max_depth=4,
 model__min_child_weight=5, model__n_estimators=600, model__reg_alpha=0.5,
 model__reg_lambda=20, model__subsample=0.9; total time= 12.6s
 [CV] END model__colsample_bylevel=0.7, model__colsample_bytree=0.8,
 model__gamma=1, model__learning_rate=0.03, model__max_depth=4,
 model__min_child_weight=5, model__n_estimators=600, model__reg_alpha=0.5,
 model__reg_lambda=20, model__subsample=0.9; total time= 11.2s
 [CV] END model__colsample_bylevel=0.7, model__colsample_bytree=0.8,
 model__gamma=1, model__learning_rate=0.03, model__max_depth=4,
 model__min_child_weight=5, model__n_estimators=600, model__reg_alpha=0.5,
 model__reg_lambda=20, model__subsample=0.9; total time= 12.7s
 [CV] END model__colsample_bylevel=0.6, model__colsample_bytree=0.6,
 model__gamma=0.1, model__learning_rate=0.03, model__max_depth=4,
 model__min_child_weight=5, model__n_estimators=600, model__reg_alpha=0,
 model__reg_lambda=10, model__subsample=0.7; total time= 13.1s
 [CV] END model__colsample_bylevel=0.6, model__colsample_bytree=0.6,
 model__gamma=0.1, model__learning_rate=0.03, model__max_depth=4,
 model__min_child_weight=5, model__n_estimators=600, model__reg_alpha=0,
 model__reg_lambda=10, model__subsample=0.7; total time= 19.4s
 [CV] END model__colsample_bylevel=0.6, model__colsample_bytree=0.6,
 model__gamma=0.1, model__learning_rate=0.03, model__max_depth=4,
 model__min_child_weight=5, model__n_estimators=600, model__reg_alpha=0,
 model__reg_lambda=10, model__subsample=0.7; total time= 15.0s
 [CV] END model__colsample_bylevel=0.6, model__colsample_bytree=0.6,
 model__gamma=0.1, model__learning_rate=0.03, model__max_depth=4,
 model__min_child_weight=5, model__n_estimators=600, model__reg_alpha=0,
 model__reg_lambda=10, model__subsample=0.7; total time= 12.2s
 [CV] END model__colsample_bylevel=0.6, model__colsample_bytree=0.6,
 model__gamma=0.1, model__learning_rate=0.03, model__max_depth=4,
 model__min_child_weight=5, model__n_estimators=600, model__reg_alpha=0,
 model__reg_lambda=10, model__subsample=0.7; total time= 17.7s
 [CV] END model__colsample_bylevel=0.8, model__colsample_bytree=0.6,
 model__gamma=0.5, model__learning_rate=0.05, model__max_depth=5,
 model__min_child_weight=10, model__n_estimators=1000, model__reg_alpha=2,
 model__reg_lambda=10, model__subsample=0.6; total time= 20.3s
 [CV] END model__colsample_bylevel=0.8, model__colsample_bytree=0.6,
 model__gamma=0.5, model__learning_rate=0.05, model__max_depth=5,
 model__min_child_weight=10, model__n_estimators=1000, model__reg_alpha=2,

```

model__reg_lambda=10, model__subsample=0.6; total time= 19.1s
[CV] END model__colsample_bylevel=0.8, model__colsample_bytree=0.6,
model__gamma=0.5, model__learning_rate=0.05, model__max_depth=5,
model__min_child_weight=10, model__n_estimators=1000, model__reg_alpha=2,
model__reg_lambda=10, model__subsample=0.6; total time= 19.4s
[CV] END model__colsample_bylevel=0.8, model__colsample_bytree=0.6,
model__gamma=0.5, model__learning_rate=0.05, model__max_depth=5,
model__min_child_weight=10, model__n_estimators=1000, model__reg_alpha=2,
model__reg_lambda=10, model__subsample=0.6; total time= 19.5s
[CV] END model__colsample_bylevel=0.6, model__colsample_bytree=0.7,
model__gamma=0, model__learning_rate=0.03, model__max_depth=3,
model__min_child_weight=3, model__n_estimators=800, model__reg_alpha=0.5,
model__reg_lambda=10, model__subsample=0.6; total time= 12.3s
[CV] END model__colsample_bylevel=0.6, model__colsample_bytree=0.7,
model__gamma=0, model__learning_rate=0.03, model__max_depth=3,
model__min_child_weight=3, model__n_estimators=800, model__reg_alpha=0.5,
model__reg_lambda=10, model__subsample=0.6; total time= 16.5s
[CV] END model__colsample_bylevel=0.8, model__colsample_bytree=0.6,
model__gamma=0.5, model__learning_rate=0.05, model__max_depth=5,
model__min_child_weight=10, model__n_estimators=1000, model__reg_alpha=2,
model__reg_lambda=10, model__subsample=0.6; total time= 21.2s
[CV] END model__colsample_bylevel=0.6, model__colsample_bytree=0.7,
model__gamma=0, model__learning_rate=0.03, model__max_depth=3,
model__min_child_weight=3, model__n_estimators=800, model__reg_alpha=0.5,
model__reg_lambda=10, model__subsample=0.6; total time= 23.0s
[CV] END model__colsample_bylevel=0.6, model__colsample_bytree=0.4,
model__gamma=0, model__learning_rate=0.03, model__max_depth=7,
model__min_child_weight=1, model__n_estimators=200, model__reg_alpha=0.5,
model__reg_lambda=5, model__subsample=1.0; total time= 7.3s
[CV] END model__colsample_bylevel=0.6, model__colsample_bytree=0.7,
model__gamma=0, model__learning_rate=0.03, model__max_depth=3,
model__min_child_weight=3, model__n_estimators=800, model__reg_alpha=0.5,
model__reg_lambda=10, model__subsample=0.6; total time= 17.8s
[CV] END model__colsample_bylevel=0.6, model__colsample_bytree=0.4,
model__gamma=0, model__learning_rate=0.03, model__max_depth=7,
model__min_child_weight=1, model__n_estimators=200, model__reg_alpha=0.5,
model__reg_lambda=5, model__subsample=1.0; total time= 8.9s
[CV] END model__colsample_bylevel=0.6, model__colsample_bytree=0.7,
model__gamma=0, model__learning_rate=0.03, model__max_depth=3,
model__min_child_weight=3, model__n_estimators=800, model__reg_alpha=0.5,
model__reg_lambda=10, model__subsample=0.6; total time= 14.1s
[CV] END model__colsample_bylevel=0.6, model__colsample_bytree=0.4,
model__gamma=0, model__learning_rate=0.03, model__max_depth=7,
model__min_child_weight=1, model__n_estimators=200, model__reg_alpha=0.5,
model__reg_lambda=5, model__subsample=1.0; total time= 9.9s
[CV] END model__colsample_bylevel=0.6, model__colsample_bytree=0.4,
model__gamma=0, model__learning_rate=0.03, model__max_depth=7,
model__min_child_weight=1, model__n_estimators=200, model__reg_alpha=0.5,

```

```

model__reg_lambda=5, model__subsample=1.0; total time= 8.4s
[CV] END model__colsample_bylevel=0.6, model__colsample_bytree=0.4,
model__gamma=0, model__learning_rate=0.03, model__max_depth=7,
model__min_child_weight=1, model__n_estimators=200, model__reg_alpha=0.5,
model__reg_lambda=5, model__subsample=1.0; total time= 8.8s
[CV] END model__colsample_bylevel=0.8, model__colsample_bytree=0.5,
model__gamma=0, model__learning_rate=0.05, model__max_depth=5,
model__min_child_weight=3, model__n_estimators=200, model__reg_alpha=0.1,
model__reg_lambda=0.5, model__subsample=0.6; total time= 8.0s
[CV] END model__colsample_bylevel=0.8, model__colsample_bytree=0.5,
model__gamma=0, model__learning_rate=0.05, model__max_depth=5,
model__min_child_weight=3, model__n_estimators=200, model__reg_alpha=0.1,
model__reg_lambda=0.5, model__subsample=0.6; total time= 5.9s
[CV] END model__colsample_bylevel=0.8, model__colsample_bytree=0.5,
model__gamma=0, model__learning_rate=0.05, model__max_depth=5,
model__min_child_weight=3, model__n_estimators=200, model__reg_alpha=0.1,
model__reg_lambda=0.5, model__subsample=0.6; total time= 6.7s
[CV] END model__colsample_bylevel=0.8, model__colsample_bytree=0.5,
model__gamma=0, model__learning_rate=0.05, model__max_depth=5,
model__min_child_weight=3, model__n_estimators=200, model__reg_alpha=0.1,
model__reg_lambda=0.5, model__subsample=0.6; total time= 7.8s
[CV] END model__colsample_bylevel=0.8, model__colsample_bytree=0.5,
model__gamma=0, model__learning_rate=0.05, model__max_depth=5,
model__min_child_weight=3, model__n_estimators=200, model__reg_alpha=0.1,
model__reg_lambda=0.5, model__subsample=0.6; total time= 9.5s
[CV] END model__colsample_bylevel=0.5, model__colsample_bytree=0.5,
model__gamma=1, model__learning_rate=0.1, model__max_depth=6,
model__min_child_weight=5, model__n_estimators=800, model__reg_alpha=0,
model__reg_lambda=1, model__subsample=1.0; total time= 14.8s
[CV] END model__colsample_bylevel=0.5, model__colsample_bytree=0.5,
model__gamma=1, model__learning_rate=0.1, model__max_depth=6,
model__min_child_weight=5, model__n_estimators=800, model__reg_alpha=0,
model__reg_lambda=1, model__subsample=1.0; total time= 17.1s
[CV] END model__colsample_bylevel=0.5, model__colsample_bytree=0.5,
model__gamma=1, model__learning_rate=0.1, model__max_depth=6,
model__min_child_weight=5, model__n_estimators=800, model__reg_alpha=0,
model__reg_lambda=1, model__subsample=1.0; total time= 13.4s
[CV] END model__colsample_bylevel=0.5, model__colsample_bytree=0.5,
model__gamma=1, model__learning_rate=0.1, model__max_depth=6,
model__min_child_weight=5, model__n_estimators=800, model__reg_alpha=0,
model__reg_lambda=1, model__subsample=1.0; total time= 14.9s
[CV] END model__colsample_bylevel=0.5, model__colsample_bytree=0.5,
model__gamma=1, model__learning_rate=0.1, model__max_depth=6,
model__min_child_weight=5, model__n_estimators=800, model__reg_alpha=0,
model__reg_lambda=1, model__subsample=1.0; total time= 15.7s
[CV] END model__colsample_bylevel=0.4, model__colsample_bytree=0.4,
model__gamma=0, model__learning_rate=0.1, model__max_depth=5,
model__min_child_weight=5, model__n_estimators=1000, model__reg_alpha=0.1,

```



```
model__reg_lambda=0.5, model__subsample=1.0; total time= 26.3s  
[CV] END model__colsample_bylevel=0.4, model__colsample_bytrees=0.4,  
model__gamma=0, model__learning_rate=0.1, model__max_depth=5,  
model__min_child_weight=5, model__n_estimators=1000, model__reg_alpha=0.1,  
model__reg_lambda=0.5, model__subsample=1.0; total time= 25.3s  
[CV] END model__colsample_bylevel=0.4, model__colsample_bytrees=0.4,  
model__gamma=0, model__learning_rate=0.1, model__max_depth=5,  
model__min_child_weight=5, model__n_estimators=1000, model__reg_alpha=0.1,  
model__reg_lambda=0.5, model__subsample=1.0; total time= 27.7s  
[CV] END model__colsample_bylevel=0.4, model__colsample_bytrees=0.4,  
model__gamma=0, model__learning_rate=0.1, model__max_depth=5,  
model__min_child_weight=5, model__n_estimators=1000, model__reg_alpha=0.1,  
model__reg_lambda=0.5, model__subsample=1.0; total time= 19.5s  
[CV] END model__colsample_bylevel=0.6, model__colsample_bytrees=0.4,  
model__gamma=1, model__learning_rate=0.1, model__max_depth=5,  
model__min_child_weight=7, model__n_estimators=600, model__reg_alpha=0.01,  
model__reg_lambda=1, model__subsample=0.7; total time= 12.5s  
[CV] END model__colsample_bylevel=0.6, model__colsample_bytrees=0.4,  
model__gamma=1, model__learning_rate=0.1, model__max_depth=5,  
model__min_child_weight=7, model__n_estimators=600, model__reg_alpha=0.01,  
model__reg_lambda=1, model__subsample=0.7; total time= 9.1s  
[CV] END model__colsample_bylevel=0.6, model__colsample_bytrees=0.4,  
model__gamma=1, model__learning_rate=0.1, model__max_depth=5,  
model__min_child_weight=7, model__n_estimators=600, model__reg_alpha=0.01,  
model__reg_lambda=1, model__subsample=0.7; total time= 11.7s  
[CV] END model__colsample_bylevel=0.6, model__colsample_bytrees=0.4,  
model__gamma=1, model__learning_rate=0.1, model__max_depth=5,  
model__min_child_weight=7, model__n_estimators=600, model__reg_alpha=0.01,  
model__reg_lambda=1, model__subsample=0.7; total time= 10.7s  
[CV] END model__colsample_bylevel=0.4, model__colsample_bytrees=0.4,  
model__gamma=0, model__learning_rate=0.1, model__max_depth=5,  
model__min_child_weight=5, model__n_estimators=1000, model__reg_alpha=0.1,  
model__reg_lambda=0.5, model__subsample=1.0; total time= 23.8s  
[CV] END model__colsample_bylevel=0.6, model__colsample_bytrees=0.4,  
model__gamma=1, model__learning_rate=0.1, model__max_depth=5,  
model__min_child_weight=7, model__n_estimators=600, model__reg_alpha=0.01,  
model__reg_lambda=1, model__subsample=0.7; total time= 9.4s  
[CV] END model__colsample_bylevel=0.4, model__colsample_bytrees=0.8,  
model__gamma=0, model__learning_rate=0.1, model__max_depth=4,  
model__min_child_weight=1, model__n_estimators=1000, model__reg_alpha=0,  
model__reg_lambda=0.5, model__subsample=1.0; total time= 23.4s  
[CV] END model__colsample_bylevel=0.4, model__colsample_bytrees=0.8,  
model__gamma=0, model__learning_rate=0.1, model__max_depth=4,  
model__min_child_weight=1, model__n_estimators=1000, model__reg_alpha=0,  
model__reg_lambda=0.5, model__subsample=1.0; total time= 20.9s  
[CV] END model__colsample_bylevel=0.4, model__colsample_bytrees=0.8,  
model__gamma=0, model__learning_rate=0.1, model__max_depth=4,  
model_min child weight=1, model_n estimators=1000, model_reg alpha=0,
```

```

model__reg_lambda=0.5, model__subsample=1.0; total time= 26.2s
[CV] END model__colsample_bylevel=0.4, model__colsample_bytree=0.8,
model__gamma=0, model__learning_rate=0.1, model__max_depth=4,
model__min_child_weight=1, model__n_estimators=1000, model__reg_alpha=0,
model__reg_lambda=0.5, model__subsample=1.0; total time= 24.4s
[CV] END model__colsample_bylevel=0.8, model__colsample_bytree=0.7,
model__gamma=0, model__learning_rate=0.03, model__max_depth=4,
model__min_child_weight=3, model__n_estimators=400, model__reg_alpha=2,
model__reg_lambda=10, model__subsample=0.6; total time= 9.7s
[CV] END model__colsample_bylevel=0.4, model__colsample_bytree=0.8,
model__gamma=0, model__learning_rate=0.1, model__max_depth=4,
model__min_child_weight=1, model__n_estimators=1000, model__reg_alpha=0,
model__reg_lambda=0.5, model__subsample=1.0; total time= 21.0s
[CV] END model__colsample_bylevel=0.8, model__colsample_bytree=0.7,
model__gamma=0, model__learning_rate=0.03, model__max_depth=4,
model__min_child_weight=3, model__n_estimators=400, model__reg_alpha=2,
model__reg_lambda=10, model__subsample=0.6; total time= 12.8s
[CV] END model__colsample_bylevel=0.8, model__colsample_bytree=0.7,
model__gamma=0, model__learning_rate=0.03, model__max_depth=4,
model__min_child_weight=3, model__n_estimators=400, model__reg_alpha=2,
model__reg_lambda=10, model__subsample=0.6; total time= 15.2s
[CV] END model__colsample_bylevel=0.8, model__colsample_bytree=0.7,
model__gamma=0, model__learning_rate=0.03, model__max_depth=4,
model__min_child_weight=3, model__n_estimators=400, model__reg_alpha=2,
model__reg_lambda=10, model__subsample=0.6; total time= 13.8s
[CV] END model__colsample_bylevel=0.8, model__colsample_bytree=0.7,
model__gamma=0.3, model__learning_rate=0.07, model__max_depth=7,
model__min_child_weight=5, model__n_estimators=200, model__reg_alpha=0.5,
model__reg_lambda=20, model__subsample=1.0; total time= 8.3s
[CV] END model__colsample_bylevel=0.8, model__colsample_bytree=0.7,
model__gamma=0.3, model__learning_rate=0.07, model__max_depth=7,
model__min_child_weight=5, model__n_estimators=200, model__reg_alpha=0.5,
model__reg_lambda=20, model__subsample=1.0; total time= 7.5s
[CV] END model__colsample_bylevel=0.8, model__colsample_bytree=0.7,
model__gamma=0.3, model__learning_rate=0.07, model__max_depth=7,
model__min_child_weight=5, model__n_estimators=200, model__reg_alpha=0.5,
model__reg_lambda=20, model__subsample=1.0; total time= 6.6s
[CV] END model__colsample_bylevel=0.8, model__colsample_bytree=0.7,
model__gamma=0, model__learning_rate=0.03, model__max_depth=4,
model__min_child_weight=3, model__n_estimators=400, model__reg_alpha=2,
model__reg_lambda=10, model__subsample=0.6; total time= 13.5s
[CV] END model__colsample_bylevel=0.8, model__colsample_bytree=0.7,
model__gamma=0.3, model__learning_rate=0.07, model__max_depth=7,
model__min_child_weight=5, model__n_estimators=200, model__reg_alpha=0.5,
model__reg_lambda=20, model__subsample=1.0; total time= 7.2s
[CV] END model__colsample_bylevel=0.8, model__colsample_bytree=0.7,
model__gamma=0.3, model__learning_rate=0.07, model__max_depth=7,
model__min_child_weight=5, model__n_estimators=200, model__reg_alpha=0.5,

```

```

model__reg_lambda=20, model__subsample=1.0; total time= 9.9s
[CV] END model__colsample_bylevel=0.7, model__colsample_bytree=0.6,
model__gamma=0.3, model__learning_rate=0.1, model__max_depth=6,
model__min_child_weight=1, model__n_estimators=400, model__reg_alpha=0.01,
model__reg_lambda=1, model__subsample=0.6; total time= 10.7s
[CV] END model__colsample_bylevel=0.7, model__colsample_bytree=0.6,
model__gamma=0.3, model__learning_rate=0.1, model__max_depth=6,
model__min_child_weight=1, model__n_estimators=400, model__reg_alpha=0.01,
model__reg_lambda=1, model__subsample=0.6; total time= 11.7s
[CV] END model__colsample_bylevel=0.7, model__colsample_bytree=0.6,
model__gamma=0.3, model__learning_rate=0.1, model__max_depth=6,
model__min_child_weight=1, model__n_estimators=400, model__reg_alpha=0.01,
model__reg_lambda=1, model__subsample=0.6; total time= 7.8s
[CV] END model__colsample_bylevel=0.7, model__colsample_bytree=0.6,
model__gamma=0.3, model__learning_rate=0.1, model__max_depth=6,
model__min_child_weight=1, model__n_estimators=400, model__reg_alpha=0.01,
model__reg_lambda=1, model__subsample=0.6; total time= 11.3s
[CV] END model__colsample_bylevel=0.7, model__colsample_bytree=0.6,
model__gamma=0.3, model__learning_rate=0.1, model__max_depth=6,
model__min_child_weight=1, model__n_estimators=400, model__reg_alpha=0.01,
model__reg_lambda=1, model__subsample=0.6; total time= 12.6s
[CV] END model__colsample_bylevel=0.5, model__colsample_bytree=0.4,
model__gamma=0, model__learning_rate=0.1, model__max_depth=5,
model__min_child_weight=5, model__n_estimators=1000, model__reg_alpha=0.01,
model__reg_lambda=5, model__subsample=0.9; total time= 27.2s
[CV] END model__colsample_bylevel=0.5, model__colsample_bytree=0.4,
model__gamma=0, model__learning_rate=0.1, model__max_depth=5,
model__min_child_weight=5, model__n_estimators=1000, model__reg_alpha=0.01,
model__reg_lambda=5, model__subsample=0.9; total time= 27.6s
[CV] END model__colsample_bylevel=0.5, model__colsample_bytree=0.4,
model__gamma=0, model__learning_rate=0.1, model__max_depth=5,
model__min_child_weight=5, model__n_estimators=1000, model__reg_alpha=0.01,
model__reg_lambda=5, model__subsample=0.9; total time= 31.8s
[CV] END model__colsample_bylevel=0.5, model__colsample_bytree=0.4,
model__gamma=0, model__learning_rate=0.1, model__max_depth=5,
model__min_child_weight=5, model__n_estimators=1000, model__reg_alpha=0.01,
model__reg_lambda=5, model__subsample=0.9; total time= 25.2s
[CV] END model__colsample_bylevel=0.7, model__colsample_bytree=0.8,
model__gamma=0, model__learning_rate=0.01, model__max_depth=5,
model__min_child_weight=10, model__n_estimators=200, model__reg_alpha=0.01,
model__reg_lambda=5, model__subsample=0.7; total time= 10.2s
[CV] END model__colsample_bylevel=0.7, model__colsample_bytree=0.8,
model__gamma=0, model__learning_rate=0.01, model__max_depth=5,
model__min_child_weight=10, model__n_estimators=200, model__reg_alpha=0.01,
model__reg_lambda=5, model__subsample=0.7; total time= 10.9s
[CV] END model__colsample_bylevel=0.7, model__colsample_bytree=0.8,
model__gamma=0, model__learning_rate=0.01, model__max_depth=5,
model__min_child_weight=10, model__n_estimators=200, model__reg_alpha=0.01,

```

```

model__reg_lambda=5, model__subsample=0.7; total time= 11.0s
[CV] END model__colsample_bylevel=0.7, model__colsample_bytree=0.8,
model__gamma=0, model__learning_rate=0.01, model__max_depth=5,
model__min_child_weight=10, model__n_estimators=200, model__reg_alpha=0.01,
model__reg_lambda=5, model__subsample=0.7; total time= 10.2s
[CV] END model__colsample_bylevel=0.7, model__colsample_bytree=0.8,
model__gamma=0, model__learning_rate=0.01, model__max_depth=5,
model__min_child_weight=10, model__n_estimators=200, model__reg_alpha=0.01,
model__reg_lambda=5, model__subsample=0.7; total time= 10.8s
[CV] END model__colsample_bylevel=0.5, model__colsample_bytree=0.4,
model__gamma=0, model__learning_rate=0.1, model__max_depth=5,
model__min_child_weight=5, model__n_estimators=1000, model__reg_alpha=0.01,
model__reg_lambda=5, model__subsample=0.9; total time= 28.3s
[CV] END model__colsample_bylevel=0.5, model__colsample_bytree=0.4,
model__gamma=0.5, model__learning_rate=0.05, model__max_depth=3,
model__min_child_weight=1, model__n_estimators=800, model__reg_alpha=0.5,
model__reg_lambda=20, model__subsample=0.7; total time= 17.4s
[CV] END model__colsample_bylevel=0.5, model__colsample_bytree=0.4,
model__gamma=0.5, model__learning_rate=0.05, model__max_depth=3,
model__min_child_weight=1, model__n_estimators=800, model__reg_alpha=0.5,
model__reg_lambda=20, model__subsample=0.7; total time= 18.9s
[CV] END model__colsample_bylevel=0.5, model__colsample_bytree=0.4,
model__gamma=0.5, model__learning_rate=0.05, model__max_depth=3,
model__min_child_weight=1, model__n_estimators=800, model__reg_alpha=0.5,
model__reg_lambda=20, model__subsample=0.7; total time= 18.8s
[CV] END model__colsample_bylevel=0.5, model__colsample_bytree=0.4,
model__gamma=0.5, model__learning_rate=0.05, model__max_depth=3,
model__min_child_weight=1, model__n_estimators=800, model__reg_alpha=0.5,
model__reg_lambda=20, model__subsample=0.7; total time= 20.8s
[CV] END model__colsample_bylevel=0.5, model__colsample_bytree=0.4,
model__gamma=0.5, model__learning_rate=0.05, model__max_depth=3,
model__min_child_weight=1, model__n_estimators=800, model__reg_alpha=0.5,
model__reg_lambda=20, model__subsample=0.7; total time= 16.7s
[CV] END model__colsample_bylevel=0.8, model__colsample_bytree=0.7,
model__gamma=0.3, model__learning_rate=0.01, model__max_depth=4,
model__min_child_weight=7, model__n_estimators=800, model__reg_alpha=0.1,
model__reg_lambda=0.5, model__subsample=0.8; total time= 20.7s
[CV] END model__colsample_bylevel=0.8, model__colsample_bytree=0.7,
model__gamma=0.3, model__learning_rate=0.01, model__max_depth=4,
model__min_child_weight=7, model__n_estimators=800, model__reg_alpha=0.1,
model__reg_lambda=0.5, model__subsample=0.8; total time= 15.9s
[CV] END model__colsample_bylevel=0.8, model__colsample_bytree=0.7,
model__gamma=0.3, model__learning_rate=0.01, model__max_depth=4,
model__min_child_weight=7, model__n_estimators=800, model__reg_alpha=0.1,
model__reg_lambda=0.5, model__subsample=0.8; total time= 21.5s
[CV] END model__colsample_bylevel=0.8, model__colsample_bytree=0.7,
model__gamma=0.3, model__learning_rate=0.01, model__max_depth=4,
model__min_child_weight=7, model__n_estimators=800, model__reg_alpha=0.1,

```

model__reg_lambda=0.5, model__subsample=0.8; total time= 30.6s
 [CV] END model__colsample_bylevel=0.6, model__colsample_bytree=0.5,
 model__gamma=0.5, model__learning_rate=0.07, model__max_depth=4,
 model__min_child_weight=5, model__n_estimators=1000, model__reg_alpha=1,
 model__reg_lambda=20, model__subsample=0.6; total time= 19.0s
 [CV] END model__colsample_bylevel=0.6, model__colsample_bytree=0.5,
 model__gamma=0.5, model__learning_rate=0.07, model__max_depth=4,
 model__min_child_weight=5, model__n_estimators=1000, model__reg_alpha=1,
 model__reg_lambda=20, model__subsample=0.6; total time= 18.1s
 [CV] END model__colsample_bylevel=0.8, model__colsample_bytree=0.7,
 model__gamma=0.3, model__learning_rate=0.01, model__max_depth=4,
 model__min_child_weight=7, model__n_estimators=800, model__reg_alpha=0.1,
 model__reg_lambda=0.5, model__subsample=0.8; total time= 23.1s
 [CV] END model__colsample_bylevel=0.6, model__colsample_bytree=0.5,
 model__gamma=0.5, model__learning_rate=0.07, model__max_depth=4,
 model__min_child_weight=5, model__n_estimators=1000, model__reg_alpha=1,
 model__reg_lambda=20, model__subsample=0.6; total time= 20.3s
 [CV] END model__colsample_bylevel=0.8, model__colsample_bytree=0.7,
 model__gamma=0.1, model__learning_rate=0.1, model__max_depth=5,
 model__min_child_weight=5, model__n_estimators=200, model__reg_alpha=0,
 model__reg_lambda=5, model__subsample=1.0; total time= 6.3s
 [CV] END model__colsample_bylevel=0.8, model__colsample_bytree=0.7,
 model__gamma=0.1, model__learning_rate=0.1, model__max_depth=5,
 model__min_child_weight=5, model__n_estimators=200, model__reg_alpha=0,
 model__reg_lambda=5, model__subsample=1.0; total time= 7.8s
 [CV] END model__colsample_bylevel=0.8, model__colsample_bytree=0.7,
 model__gamma=0.1, model__learning_rate=0.1, model__max_depth=5,
 model__min_child_weight=5, model__n_estimators=200, model__reg_alpha=0,
 model__reg_lambda=5, model__subsample=1.0; total time= 7.4s
 [CV] END model__colsample_bylevel=0.8, model__colsample_bytree=0.7,
 model__gamma=0.1, model__learning_rate=0.1, model__max_depth=5,
 model__min_child_weight=5, model__n_estimators=200, model__reg_alpha=0,
 model__reg_lambda=5, model__subsample=1.0; total time= 5.8s
 [CV] END model__colsample_bylevel=0.6, model__colsample_bytree=0.5,
 model__gamma=0.5, model__learning_rate=0.07, model__max_depth=4,
 model__min_child_weight=5, model__n_estimators=1000, model__reg_alpha=1,
 model__reg_lambda=20, model__subsample=0.6; total time= 19.6s
 [CV] END model__colsample_bylevel=0.6, model__colsample_bytree=0.5,
 model__gamma=0.5, model__learning_rate=0.07, model__max_depth=4,
 model__min_child_weight=5, model__n_estimators=1000, model__reg_alpha=1,
 model__reg_lambda=20, model__subsample=0.6; total time= 19.4s
 [CV] END model__colsample_bylevel=0.8, model__colsample_bytree=0.7,
 model__gamma=0.1, model__learning_rate=0.1, model__max_depth=5,
 model__min_child_weight=5, model__n_estimators=200, model__reg_alpha=0,
 model__reg_lambda=5, model__subsample=1.0; total time= 8.3s
 [CV] END model__colsample_bylevel=0.6, model__colsample_bytree=0.8,
 model__gamma=0, model__learning_rate=0.1, model__max_depth=3,
 model__min_child_weight=10, model__n_estimators=600, model__reg_alpha=1,

```

model__reg_lambda=5, model__subsample=0.9; total time= 16.3s
[CV] END model__colsample_bylevel=0.6, model__colsample_bytree=0.8,
model__gamma=0, model__learning_rate=0.1, model__max_depth=3,
model__min_child_weight=10, model__n_estimators=600, model__reg_alpha=1,
model__reg_lambda=5, model__subsample=0.9; total time= 15.8s
[CV] END model__colsample_bylevel=0.6, model__colsample_bytree=0.8,
model__gamma=0, model__learning_rate=0.1, model__max_depth=3,
model__min_child_weight=10, model__n_estimators=600, model__reg_alpha=1,
model__reg_lambda=5, model__subsample=0.9; total time= 14.5s
[CV] END model__colsample_bylevel=0.6, model__colsample_bytree=0.8,
model__gamma=0, model__learning_rate=0.1, model__max_depth=3,
model__min_child_weight=10, model__n_estimators=600, model__reg_alpha=1,
model__reg_lambda=5, model__subsample=0.9; total time= 16.5s
[CV] END model__colsample_bylevel=0.5, model__colsample_bytree=0.6,
model__gamma=0.1, model__learning_rate=0.05, model__max_depth=4,
model__min_child_weight=1, model__n_estimators=600, model__reg_alpha=1,
model__reg_lambda=5, model__subsample=1.0; total time= 14.8s
[CV] END model__colsample_bylevel=0.6, model__colsample_bytree=0.8,
model__gamma=0, model__learning_rate=0.1, model__max_depth=3,
model__min_child_weight=10, model__n_estimators=600, model__reg_alpha=1,
model__reg_lambda=5, model__subsample=0.9; total time= 16.6s
[CV] END model__colsample_bylevel=0.5, model__colsample_bytree=0.6,
model__gamma=0.1, model__learning_rate=0.05, model__max_depth=4,
model__min_child_weight=1, model__n_estimators=600, model__reg_alpha=1,
model__reg_lambda=5, model__subsample=1.0; total time= 13.7s
[CV] END model__colsample_bylevel=0.5, model__colsample_bytree=0.6,
model__gamma=0.1, model__learning_rate=0.05, model__max_depth=4,
model__min_child_weight=1, model__n_estimators=600, model__reg_alpha=1,
model__reg_lambda=5, model__subsample=1.0; total time= 14.6s
[CV] END model__colsample_bylevel=0.5, model__colsample_bytree=0.6,
model__gamma=0.1, model__learning_rate=0.05, model__max_depth=4,
model__min_child_weight=1, model__n_estimators=600, model__reg_alpha=1,
model__reg_lambda=5, model__subsample=1.0; total time= 9.7s
[CV] END model__colsample_bylevel=0.5, model__colsample_bytree=0.6,
model__gamma=0.1, model__learning_rate=0.01, model__max_depth=5,
model__min_child_weight=10, model__n_estimators=400, model__reg_alpha=0,
model__reg_lambda=1, model__subsample=1.0; total time= 15.2s
[CV] END model__colsample_bylevel=0.5, model__colsample_bytree=0.6,
model__gamma=0.1, model__learning_rate=0.05, model__max_depth=4,
model__min_child_weight=1, model__n_estimators=600, model__reg_alpha=1,
model__reg_lambda=5, model__subsample=1.0; total time= 16.2s
[CV] END model__colsample_bylevel=0.5, model__colsample_bytree=0.6,
model__gamma=0.1, model__learning_rate=0.01, model__max_depth=5,
model__min_child_weight=10, model__n_estimators=400, model__reg_alpha=0,
model__reg_lambda=1, model__subsample=1.0; total time= 19.8s
[CV] END model__colsample_bylevel=0.5, model__colsample_bytree=0.6,
model__gamma=0.1, model__learning_rate=0.01, model__max_depth=5,
model__min_child_weight=10, model__n_estimators=400, model__reg_alpha=0,

```

```
model__reg_lambda=1, model__subsampling=1.0; total time= 17.3s  
[CV] END model__colsamples_by_level=0.5, model__colsamples_by_tree=0.6,  
model__gamma=0.1, model__learning_rate=0.01, model__max_depth=5,  
model__min_child_weight=10, model__n_estimators=400, model__reg_alpha=0,  
model__reg_lambda=1, model__subsampling=1.0; total time= 17.7s  
[CV] END model__colsamples_by_level=0.5, model__colsamples_by_tree=0.6,  
model__gamma=0.1, model__learning_rate=0.01, model__max_depth=5,  
model__min_child_weight=10, model__n_estimators=400, model__reg_alpha=0,  
model__reg_lambda=1, model__subsampling=1.0; total time= 17.8s  
[CV] END model__colsamples_by_level=0.5, model__colsamples_by_tree=0.6,  
model__gamma=0.1, model__learning_rate=0.01, model__max_depth=5,  
model__min_child_weight=7, model__n_estimators=1000, model__reg_alpha=0,  
model__reg_lambda=1, model__subsampling=0.8; total time= 23.9s  
[CV] END model__colsamples_by_level=0.5, model__colsamples_by_tree=0.6,  
model__gamma=0.1, model__learning_rate=0.01, model__max_depth=5,  
model__min_child_weight=7, model__n_estimators=1000, model__reg_alpha=0,  
model__reg_lambda=1, model__subsampling=0.8; total time= 31.1s  
[CV] END model__colsamples_by_level=0.5, model__colsamples_by_tree=0.6,  
model__gamma=0.1, model__learning_rate=0.01, model__max_depth=5,  
model__min_child_weight=7, model__n_estimators=1000, model__reg_alpha=0,  
model__reg_lambda=1, model__subsampling=0.8; total time= 31.1s  
[CV] END model__colsamples_by_level=0.5, model__colsamples_by_tree=0.6,  
model__gamma=0.1, model__learning_rate=0.01, model__max_depth=5,  
model__min_child_weight=7, model__n_estimators=1000, model__reg_alpha=0,  
model__reg_lambda=1, model__subsampling=0.8; total time= 31.5s  
[CV] END model__colsamples_by_level=0.8, model__colsamples_by_tree=0.7,  
model__gamma=1, model__learning_rate=0.03, model__max_depth=5,  
model__min_child_weight=5, model__n_estimators=200, model__reg_alpha=1,  
model__reg_lambda=20, model__subsampling=0.7; total time= 10.2s  
[CV] END model__colsamples_by_level=0.8, model__colsamples_by_tree=0.7,  
model__gamma=1, model__learning_rate=0.03, model__max_depth=5,  
model__min_child_weight=5, model__n_estimators=200, model__reg_alpha=1,  
model__reg_lambda=20, model__subsampling=0.7; total time= 9.2s  
[CV] END model__colsamples_by_level=0.8, model__colsamples_by_tree=0.7,  
model__gamma=1, model__learning_rate=0.03, model__max_depth=5,  
model__min_child_weight=5, model__n_estimators=200, model__reg_alpha=1,  
model__reg_lambda=20, model__subsampling=0.7; total time= 11.0s  
[CV] END model__colsamples_by_level=0.8, model__colsamples_by_tree=0.7,  
model__gamma=1, model__learning_rate=0.03, model__max_depth=5,  
model__min_child_weight=5, model__n_estimators=200, model__reg_alpha=1,  
model__reg_lambda=20, model__subsampling=0.7; total time= 8.9s  
[CV] END model__colsamples_by_level=0.8, model__colsamples_by_tree=0.7,  
model__gamma=1, model__learning_rate=0.03, model__max_depth=5,  
model__min_child_weight=5, model__n_estimators=200, model__reg_alpha=1,  
model__reg_lambda=20, model__subsampling=0.7; total time= 6.7s  
[CV] END model__colsamples_by_level=0.5, model__colsamples_by_tree=0.6,  
model__gamma=0.1, model__learning_rate=0.01, model__max_depth=5,  
model_min_child_weight=7, model_n_estimators=1000, model_reg_alpha=0,
```

```

model__reg_lambda=1, model__subsample=0.8; total time= 31.6s
[CV] END model__colsample_bylevel=0.4, model__colsample_bytree=0.7,
model__gamma=0, model__learning_rate=0.07, model__max_depth=3,
model__min_child_weight=10, model__n_estimators=400, model__reg_alpha=0.01,
model__reg_lambda=5, model__subsample=1.0; total time= 9.6s
[CV] END model__colsample_bylevel=0.4, model__colsample_bytree=0.7,
model__gamma=0, model__learning_rate=0.07, model__max_depth=3,
model__min_child_weight=10, model__n_estimators=400, model__reg_alpha=0.01,
model__reg_lambda=5, model__subsample=1.0; total time= 7.4s
[CV] END model__colsample_bylevel=0.4, model__colsample_bytree=0.7,
model__gamma=0, model__learning_rate=0.07, model__max_depth=3,
model__min_child_weight=10, model__n_estimators=400, model__reg_alpha=0.01,
model__reg_lambda=5, model__subsample=1.0; total time= 14.8s
[CV] END model__colsample_bylevel=0.4, model__colsample_bytree=0.7,
model__gamma=0, model__learning_rate=0.07, model__max_depth=3,
model__min_child_weight=10, model__n_estimators=400, model__reg_alpha=0.01,
model__reg_lambda=5, model__subsample=1.0; total time= 11.4s
[CV] END model__colsample_bylevel=0.4, model__colsample_bytree=0.7,
model__gamma=0, model__learning_rate=0.07, model__max_depth=3,
model__min_child_weight=10, model__n_estimators=400, model__reg_alpha=0.01,
model__reg_lambda=5, model__subsample=1.0; total time= 12.9s
[CV] END model__colsample_bylevel=0.4, model__colsample_bytree=0.4,
model__gamma=0, model__learning_rate=0.07, model__max_depth=6,
model__min_child_weight=1, model__n_estimators=1000, model__reg_alpha=0.01,
model__reg_lambda=20, model__subsample=0.9; total time= 31.0s
[CV] END model__colsample_bylevel=0.4, model__colsample_bytree=0.4,
model__gamma=0, model__learning_rate=0.07, model__max_depth=6,
model__min_child_weight=1, model__n_estimators=1000, model__reg_alpha=0.01,
model__reg_lambda=20, model__subsample=0.9; total time= 29.3s
[CV] END model__colsample_bylevel=0.4, model__colsample_bytree=0.4,
model__gamma=0, model__learning_rate=0.07, model__max_depth=6,
model__min_child_weight=1, model__n_estimators=1000, model__reg_alpha=0.01,
model__reg_lambda=20, model__subsample=0.9; total time= 33.8s
[CV] END model__colsample_bylevel=0.4, model__colsample_bytree=0.4,
model__gamma=0, model__learning_rate=0.07, model__max_depth=6,
model__min_child_weight=1, model__n_estimators=1000, model__reg_alpha=0.01,
model__reg_lambda=20, model__subsample=0.9; total time= 32.8s
[CV] END model__colsample_bylevel=0.4, model__colsample_bytree=0.4,
model__gamma=0, model__learning_rate=0.07, model__max_depth=6,
model__min_child_weight=1, model__n_estimators=1000, model__reg_alpha=0.01,
model__reg_lambda=20, model__subsample=0.9; total time= 18.7s

```

```

Best params: {'model__subsample': 0.9, 'model__reg_lambda': 20,
'model__reg_alpha': 0.01, 'model__n_estimators': 1000,
'model__min_child_weight': 1, 'model__max_depth': 6, 'model__learning_rate':
0.07, 'model__gamma': 0, 'model__colsample_bytree': 0.4,
'model__colsample_bylevel': 0.4}
Best CV R2: 0.9430368076406739

```



```
--- Final tuned XGBoost ---
```

```
R2 train: 0.9589
```

```
R2 test: 0.9427
```

```
MAE: 91985.12
```

```
Saved best pipeline to xgboostmodel_tuned.pkl
```

[12]:

```
['Unnamed: 0', 'car_name', 'brand', 'model', 'vehicle_age', 'km_driven',  
'seller_type', 'fuel_type', 'transmission_type', 'mileage', 'engine',  
'max_power', 'seats', 'selling_price']
```

```
--- Duplicate Checks ---
```

```
Duplicate rows in full X: 0
```

```
Duplicate rows across train+test combined: 0
```

```
--- Default RF (single split) ---
```

```
R2 train score: 0.9571
```

```
R2 test score: 0.9419
```

```
MAE: 99230.46
```

```
--- Cross-Validation (5-fold, log-space R2) ---
```

```
CV R2 scores: [0.9342 0.9177 0.9172 0.9413 0.9218]
```

```
CV R2 mean: 0.9265
```

```
CV R2 std: 0.0096
```

```
--- Tuned RF (single split) ---
```

```
R2 train score: 0.7646
```

```
R2 test score: 0.851
```

```
MAE: 122085.29
```

```
Predicted Price (tuned RF): 2,625,246
```

```
Saved tuned RF pipeline to randommodel.pkl
```